

MLF-ICL: Adaptive malicious URL detection via multi-level feature fusion and TabDPT-based in-context learning

Lan Liu^a , Fengwei Guo^a , Weijie Liang^a, Kundi Yao^{b,*}

^a Guangdong Polytechnic Normal University, Tianhe District, Guangzhou, 510655, China

^b Faculty of Engineering and Applied Science, Ontario Tech University, Oshawa, Ontario, Canada

HIGHLIGHTS

- Enhanced DCG-BERT uses Multi-order Dynamic Feature Fusion to link character anomalies with token intent in obfuscated URLs.
- Deep semantic embeddings and Saliency Map-filtered lexical rules unite implicit data-driven patterns with expert knowledge.
- We pioneer TabDPT with ICL, shifting static parameter fitting to retrieval-augmented adaptive inference for zero-day threats.
- Tests achieve state-of-the-art F1-score (99.78%) on highly imbalanced data, outperforming CNN/LSTM under concept drift.

ARTICLE INFO

Communicated by Y. Lai

Keywords:

Malicious URL detection
Multi-view feature fusion
Lexical analysis features
Tabular foundation model

ABSTRACT

Malicious URL detection faces persistent challenges from morphological heterogeneity and frequent concept drift. Traditional deep learning models rely on static decision boundaries and require computationally expensive re-training to counter emerging threats, creating a dangerous detection lag. To bridge this research gap, we propose MLF-ICL, a novel detection framework that shifts the paradigm from static classification to an inference model that utilizes retrieval and requires no training. To enable this transition and connect raw text with tabular models, we introduce a specialized architecture designed to bridge features. First, we engineer a DCG-BERT backbone enhanced by DFF to capture microscopic character obfuscations and macroscopic token semantics. Second, we develop the M-DWFS algorithm to dynamically fuse these deep representations with lexical rules derived from expert knowledge. This highly accurate multimodal feature space allows us to pioneer the application of a Tabular Foundation Model (TabDPT) equipped with In-Context Learning (ICL). By replacing parameter updates with reasoning based on retrieval, our model dynamically adapts to new threat variants using historical context. Extensive evaluations on three benchmark datasets demonstrate that MLF-ICL achieves superior performance, including an F1 score of 99.78% on highly imbalanced data. The results validate the proposed paradigm as a highly effective solution for evolving cybersecurity challenges.

1. Introduction

While Uniform Resource Locators (URLs) serve as the fundamental navigation mechanism of the World Wide Web, their pervasive nature has simultaneously rendered them a primary vector for cyber threats. Modern attack chains, including phishing, malware distribution, and social engineering, heavily rely on deceptive links to bypass network defenses and compromise users [1,2]. The scale of this threat is escalating rapidly: the Anti-Phishing Working Group (APWG) reported over 1 million unique phishing attacks in the first quarter of 2025 alone [3].

This historic peak underscores the limitations of current defenses. The consequences are severe, with the FBI's Internet Crime Complaint Center (IC3) attributing over \$2.8 billion in losses to email-compromise schemes in 2024 [4]. As illustrated in Fig. 1, malicious actors exploit the structural complexity of these URLs to craft links that are visually indistinguishable from benign ones. For instance, attackers may employ homograph attacks (e.g., replacing 'o' with '0' to create `g00g1e.com.hk`) or manipulate subdomains to mimic trusted entities. These subtle morphological nuances effectively bypass human vigilance and static filters. This necessitates advanced automated detection mechanisms [2,5].

* Corresponding author.

Email addresses: liulan@gpnu.edu.cn (L. Liu), guofengwei@stu.gpnu.edu.cn (F. Guo), liangweijie@stu.gpnu.edu.cn (W. Liang), kundi.yao@ontariotechu.ca (K. Yao).

<https://doi.org/10.1016/j.neucom.2026.134755>

Received 26 February 2026; Received in revised form 2 July 2026; Accepted 6 August 2026

Available online 8 August 2026

0925-2312/© 2026 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

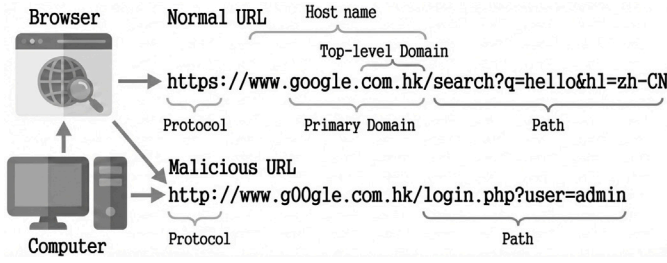


Fig. 1. URL Analysis Diagram.

Traditionally, precise and proactive URL detection relied on black-lists and handcrafted lexical features derived from domain expertise. Mamun et al. [6] demonstrated the utility of statistical properties (e.g., URL length, character distribution) for efficient classification. However, these methods struggle with newly registered domains, previously unseen phishing URLs, and rapidly changing obfuscation patterns because they depend on delayed blacklist updates or fixed expert rules. To address this, the paradigm shifted toward Deep Learning (DL). Convolutional Neural Networks (CNNs) and specialized architectures like URLNet [7], GramBeddings [8], and ensemble models [9] surpassed traditional machine learning by automatically extracting complex character-level patterns. **However, even state-of-the-art DL models suffer from a fundamental limitation: rigidity in the face of rapid concept drift.** Adversaries continuously mutate URL structures, alter top-level domains, and deploy novel obfuscation techniques, causing severe distribution shifts between offline training data and live production traffic. Conventional frameworks, operating under a rigid “train-once, deploy-forever” paradigm, cannot dynamically shift their frozen decision boundaries to accommodate these structural mutations. Advanced pre-trained language models (PLMs) [10–12] improve semantic feature extraction [13] but remain bound by static parameters post-deployment. Confronting an unobserved or zero-day-like phishing variant forces an unsustainable operational dilemma: tolerate misclassifications or execute computationally expensive pipeline retraining. This remediation cycle introduces a dangerous detection lag, leaving network infrastructure exposed during the window required to update model weights. Bridging the gap between static parameters and dynamic threat evolution requires a fundamental paradigm shift from static classification to adaptive inference.

To bridge this gap between evolving threats and static models, we contend that effective defense requires a paradigm shift from “static classification” to “adaptive inference.” Recent advancements in Tabular Foundation Models (TFMs) offer a compelling solution through **In-Context Learning (ICL)**. Unlike traditional supervision, ICL treats detection as a retrieval-augmented inference task. Models like TabPFN [14] demonstrate the potential of this approach in low-data regimes. However, TabPFN faces scalability constraints regarding context length [15]. The TabDPT [16] framework overcomes these shortcomings. It enables efficient retrieval and generalization to unseen tasks via self-supervised learning on real data. Furthermore, ICL predicts labels based on a dynamic set of retrieved “neighboring” examples rather than fixed weights. This allows for immediate adaptation to new threat distributions without parameter updates.

Despite deep learning advancements, malicious URL detection literature leaves two critical gaps unaddressed:

Gap 1: Rigidity against rapid concept drift. Current models rely on static parameters after initial training. When confronting newly created phishing domains, unseen URL obfuscation variants, or distribution shifts across phishing-oriented datasets, they suffer detection lag because

adapting to new distributions usually requires computationally expensive backpropagation. Structurally different attack families, such as DGA-based domains and DNS tunneling, further expose the boundaries of existing URL detectors and require explicit evaluation.

Gap 2: Disconnect between implicit semantics and explicit expert rules. Standard neural embeddings act as black boxes, frequently overlooking heuristic indicators like statistical entropy spikes. Conversely, traditional systems relying on rules miss latent semantic intents. The field currently lacks frameworks capable of synergistically integrating microscopic character anomalies, macroscopic token semantics, and explicit tabular expert knowledge.

To bridge these research gaps, we propose **MLF-ICL (Multi-Level Feature Fusion via In-Context Learning)**. This framework synergizes the representational power of multi-modal feature fusion with the ability of In-Context Learning to adapt without retraining. Although foundation models like CharBERT and TabDPT possess strong capabilities in their respective domains, applying them directly to cybersecurity requires overcoming significant domain shifts. Our core methodological contribution lies in bridging this gap: we propose a fundamental paradigm shift in malicious URL detection by transitioning from static classification to a retrieval-augmented, training-free inference model. To make this shift possible, we introduce specific architectural innovations tailored for cybersecurity. We design the M-DWFS algorithm and a DFF-enhanced DCG-BERT backbone. These components act as a bridging mechanism that translates morphologically complex URL strings into a highly discriminative multimodal tabular feature space. This high-fidelity feature space is what enables the TabDPT foundation model to adapt more effectively to unseen phishing-oriented distributions through context retrieval, while also allowing us to explicitly examine its boundaries under structurally different emerging threats.

The main contributions of this paper are summarized as follows:

1. **A Training-Free Adaptive Inference Paradigm:** We pioneer the application of TabDPT to malicious URL detection. We demonstrate that its retrieval-augmented ICL mechanism offers a practical alternative to static training for adapting to unseen phishing-oriented URL distributions without repetitive parameter updates. We further evaluate its boundaries under structurally different emerging threats.
2. **Synergistic Multi-Modal Feature Bridging:** We construct a robust feature space that connects raw text with tabular foundation models. By proposing a Multi-order Dynamic Feature Fusion (DFF) backbone and an M-DWFS algorithm, we successfully fuse deep semantics with expert-guided lexical statistics. This ensures the model captures both implicit intent and explicit structural anomalies.
3. **State-of-the-Art Performance:** Extensive evaluations on three benchmark datasets reveal that MLF-ICL significantly outperforms existing baselines, achieving an F1-score of 99.77% on imbalanced data. These results validate the effectiveness of combining deep representation learning with adaptive in-context inference.

Paper organization. The remainder of this paper is organized as follows. **Section 2** reviews the evolution of malicious URL detection methods and the emergence of tabular foundation models. **Section 3** introduces the benchmark datasets and their distribution characteristics. **Section 4** elaborates on the proposed MLF-ICL architecture, detailing the enhanced CharBERT backbone, the dynamic lexical feature selection mechanism, and the retrieval-augmented inference process. **Section 5** presents the experimental setup, performance comparisons, and ablation studies. Finally, **Section 6** summarizes our findings and outlines future research directions.

2. Background and related work

This section systematically reviews the developmental trajectory of malicious URL detection. We categorize related works into three evolutionary stages. The first stage involves early detection schemes based on handcrafted features and traditional machine learning. The second encompasses intermediate end-to-end deep learning architectures represented by CNNs and Transformers. Finally, the current frontier explores tabular foundation models and retrieval-augmented inference. A comprehensive comparison of representative studies is provided in.

2.1. Background on malicious URL detection

Malicious URL detection is a critical component of cybersecurity, aimed at identifying URLs that lead to harmful content, such as phishing sites, malware distribution pages, drive-by downloads, or scams. These URLs pose significant threats by exploiting user trust to steal sensitive information, install malicious software, or facilitate financial fraud.

Traditional malicious URL detection methods can be broadly classified into three categories: blacklist-based, heuristic-based, and machine learning-based approaches. Blacklist-based systems, such as Google Safe Browsing or PhishTank, maintain databases of known malicious URLs and block access upon matching. While efficient for identified threats, they fail against zero-day attacks or newly registered domains. This limitation arises because blacklists require constant updates and cannot cover the vast, dynamic web landscape. Heuristic-based methods analyze URL structures for suspicious patterns, such as unusual character distributions, excessive subdomains, or IP addresses in place of domain names. These rules are often handcrafted by experts but struggle with false positives and adaptability to evolving evasion tactics. Machine learning has emerged as a more robust paradigm, leveraging data-driven models to classify URLs based on extracted features. These features typically include lexical, host-based, and content-based categories. Classifiers like Support Vector Machines (SVM), Random Forests, or Neural Networks are trained on labeled datasets to distinguish benign from malicious URLs. However, challenges persist, such as feature engineering overhead and domain shifts between training and real-world data. Furthermore, certain methods rely too heavily on character-embedding features and neglect high-level handcrafted lexical connections; conversely, others focus solely on high-level connections while ignoring character-level information [17]. Recent advancements in deep learning and foundation models aim to address these issues. They automate feature extraction and aim to improve transferability, but their robustness under temporal drift, cross-dataset shifts, and cross-family attacks still requires explicit validation.

2.2. Malicious URL detection based on lexical features and traditional machine learning

Early research established lexical features as an effective approach for identifying malicious URLs. Recognizing that phishing domains often exhibit statistical distributions distinct from benign ones in terms of length, character combinations, and keyword usage, McGrath et al. [18] first quantified these distributional discrepancies. Building on this, Ma et al. [19,20] expanded the feature dimensions by combining lexical features with host-based features, noting that attackers often employ specific deceptive keywords to obfuscate their intent. Subsequent research focused on optimizing handcrafted feature engineering. Representative systems, such as CatchPhish [21] and those proposed by Sahingoz et al. [22], extracted multidimensional features including string length, specific vocabulary counts, and subdomain frequency. More recent studies have continued this traditional learning paradigm by benchmarking logistic regression, confirming that optimized classical models remain practical for lightweight malicious URL screening [23]. These approaches achieved high accuracy (over 97%) with classifiers like Random Forest (RF) and SVM [21–23].

However, the generalizability of these methods is often constrained by dataset-specific biases. Their robustness faces significant challenges when analyzing evolving attack patterns or cross-domain data, as they rely heavily on static, expert-defined rules.

2.3. CNN-based malicious URL detection

With the advancement of deep learning, Convolutional Neural Networks (CNNs) have been widely adopted for their superior local feature extraction capabilities. Huang et al. [24] pioneered a hybrid architecture combining convolutional layers with Capsule Networks to capture the spatial hierarchy of URLs. Wang et al. [25] explored multi-modal feature fusion, constructing a CNN-RNN hybrid model to integrate sequential features with Word2Vec-based static vocabulary features. In this progression, URLNet by Le et al. [7] stands as a significant milestone. It innovatively introduced a dual-branch CNN architecture to learn character-level and word-level embeddings, fusing them at the top layer. This “multi-granularity dual-view” design established a paradigm for feature fusion. Recently, Bozkir et al. [8] proposed GramBeddings [8], which further integrates CNNs, LSTMs, and attention mechanisms. By refining feature representation through n-gram techniques, it demonstrated performance superior to URLNet, validating the advantage of deep representation learning in automated feature extraction.

2.4. Transformer-based malicious URL detection

The success of Transformers in NLP prompted researchers to explore their application in malicious URL detection. Chang et al. [26] first attempted a transfer learning paradigm by fine-tuning a pre-trained BERT on URL data. Subsequently, Maneriker et al. [27] proposed URL-Tran, a comprehensive benchmark of various Transformer models. However, directly applying NLP models faces a significant “Domain Shift” problem, where the syntax of natural language differs fundamentally from the structural patterns of URLs. To address this, Wang et al. [28] combined Transformers with Mixture-of-Experts (MoE), while Xu et al. [29] proposed lightweight variants. A more in-depth exploration came from Wang et al., who pre-trained a domain-specific BERT from scratch for URL data. While this eliminated domain shift, it incurred high computational costs. Conversely, specialized architectures like TransURL [11], which build upon a pre-trained CharBERT [13] backbone by utilizing CNNs for local interactions and Transformers for global semantics, demonstrated that capturing local patterns at the character level is often more critical than long-range dependencies in URL detection, offering a more efficient design pathway.

2.5. Tabular foundation models & in-context learning

Recently, a new paradigm has emerged in deep learning: Tabular Foundation Models (TFMs). Their core advantage lies in In-Context Learning (ICL), enabling adaptation to new tasks via a single forward pass without traditional gradient-based fine-tuning [30]. TabPFN represents a breakthrough in this domain [31]. As a Prior-Data Fitted Network (PFN), it was pre-trained on 130 million synthetic samples to simulate Bayesian inference within a neural framework, achieving high efficiency and accuracy. However, TabPFN suffers from strict scalability limitations, being suitable only for tasks with small sample sizes (<1000) and limited features [32]. Addressing this bottleneck, TabDPT marks a significant leap in scalability. TabDPT [14] introduces a retrieval-based context mechanism, breaking input length constraints by dynamically retrieving relevant samples from massive datasets. Furthermore, its denoising self-supervised pre-training strategy enables robust feature learning from a mixture of synthetic and real-world data [33]. Despite the success of TFMs in general tabular tasks, effectively adapting this architecture, which is designed primarily for abstract numerical features, to the highly domain-specific (heterogeneous and sequential) task of

malicious URL detection remains an open challenge. This is the core problem this paper seeks to address (Table 1).

3. Malicious URL data

To evaluate our proposed model, we used three publicly available datasets that differ in sample size and features, as shown in Figs. 2 and 3. We plotted radar charts and URL length bar charts for benign and malicious samples. This provides a visual understanding of their specific characteristics. The detailed descriptions of the datasets are as follows:

1. Dataset 1 (Malicious and Benign URLs Dataset)¹: This is a standard benchmark dataset for malicious URL detection, sourced from the Kaggle competition platform. It contains 450,176 unique URL records and is intended to support the development and evaluation of machine learning classifiers for identifying cybersecurity threats. A core feature of this dataset is its significant class imbalance, which closely simulates the natural distribution of benign and malicious links in real-world scenarios. Specifically, approximately 77% (345,738) of the URLs are labeled as benign, while the remaining 23% (104,438) are identified as malicious.
2. Dataset 2 (PhiUSIIL dataset)²: The PhiUSIIL dataset is a large-scale collection of 134,850 safe URLs and 100,945 malicious ones used to train cybersecurity systems. It was developed by Prasad et al. [34] to evaluate their proposed phishing URL detection framework. To build the repository, they extracted a mix of basic URL characteristics, HTML code details from downloaded web pages, and uniquely derived metrics. Ultimately, this extensive and actively gathered data helps machine learning models adapt to and identify new, highly deceptive web threats.
3. Dataset 3 (Malicious URLs Dataset)³: This is a comprehensive URL dataset widely used in cybersecurity research and is publicly available on the Kaggle platform. Comprising 651,191 URL records, its core value lies in providing multi-class threat labels that go beyond simple binary classification. This granular taxonomy categorizes URLs into four major classes characterized by significant imbalance: 428,103 benign links, 96,457 defacement links, 94,111 phishing links, and 32,520 malware links. This dataset was constructed by aggregating data from multiple sources, including ISCX-URL-2016, PhishTank, and GitHub repositories. Its multi-class nature and high imbalance make it an ideal benchmark for effectively evaluating the precision of machine learning models in distinguishing between diverse real-world attack vectors and malicious code injection.
4. Dataset 4: This dataset contains 30,000 records, combining 10,000 randomly sampled records each from the DNS Tunneling [35], UMUDGA [36], and UTL_DGA22 [37] datasets. Specifically, we sourced the data samples for our study from the preprocessed and publicly available GitHub repository provided by Mahdaouy et al. [38]. Unlike standard phishing URLs that use visual deception to trick humans, DGA and DNS tunneling are stealthy, machine-to-machine network threats. DGA dynamically generates random domain names to evade blacklists and connect to malware command servers. Meanwhile, DNS tunneling hides stolen data or malicious commands within seemingly normal DNS queries to bypass firewalls. Ultimately, this dataset was prepared to serve as the foundation for subsequent cross-dataset experiments.

¹ <https://www.kaggle.com/datasets/siddharthkumar25/malicious-and-benign-urls>.

² <https://github.com/elaaatif/DATA-MINING-PhiUSIIL-Phishing-URL>.

³ <https://www.kaggle.com/datasets/sid321axn/malicious-urls-dataset>.

4. MLF-ICL architecture

The MLF-ICL framework addresses two challenges in malicious URL detection: *morphological heterogeneity* and *concept drift*. Morphological heterogeneity refers to diverse URL obfuscation patterns, whereas concept drift refers to changes in attack distributions over time. As shown in Fig. 4, MLF-ICL follows three main stages.

1. **Deep Semantic Extraction:** URLs are not ordinary natural-language sentences. Attackers often use character-level substitutions or abnormal token structures to imitate legitimate domains. To capture these patterns, the grammar feature pathway uses an enhanced DCG-BERT backbone with dual token and character channels.
2. **Expert Knowledge Injection:** Neural representations may miss explicit phishing indicators, such as entropy spikes, suspicious keywords, or abnormal URL structures. We therefore construct handcrafted lexical analysis features from morphological, statistical, heuristic, and information-theoretic dimensions. M-DWFS then filters and weights these features to reduce redundancy.
3. **Adaptive Inference:** Static classifiers rely on fixed decision boundaries after training. This limits their ability to adapt to new attack patterns. MLF-ICL uses TabDPT as the inference module. Given the fused grammar and lexical analysis features, TabDPT retrieves relevant historical samples and performs In-Context Learning (ICL) for classification.

Fig. 4 summarizes the full workflow. The raw URL is first encoded by the CharBERT implementation adapted by Liu et al. [12]. CharBERT is used because its dual-channel design captures both token-level semantics and character-level morphology. This design is suitable for URLs, which are short, heterogeneous, and often ungrammatical.

The DCG-BERT module aggregates multi-level token and character representations through the heterogeneous interaction module and the Dynamic Feature Fusion (DFF) mechanism. Context Gating then recalibrates the global representation, and Spatial Dropout is used for regularization. During representation learning, we use focal loss, weighted random sampling, and hard negative mining to improve robustness under class imbalance.

In parallel, lexical prior knowledge is encoded as handcrafted URL features. M-DWFS selects the most informative lexical analysis features and assigns dynamic weights to them. The grammar features and lexical analysis features are then concatenated into a multimodal tabular vector. During inference, a FAISS index retrieves historical samples nearest to the test instance. TabDPT uses these retrieved samples as context and predicts the final URL label through ICL, without parameter fine-tuning.

4.1. Data preprocessing

Before model training, each URL was converted into paired word-level and character-level inputs through the preprocessing pipeline shown in Fig. 5. For the CharBERT branch, the raw URL string was first trimmed using `strip()`, tokenized by the BERT tokenizer, and augmented with [CLS] and [SEP] markers. The token IDs were then truncated or padded to a fixed length of 200, with the corresponding `input_ids`, `input_types`, and `input_masks` generated.

In parallel, character inputs were built directly from the original URL rather than from tokenized subwords. Nonprintable characters and whitespace were removed, and each remaining character was mapped to the local character vocabulary. Unseen characters were assigned to the [UNK] index. The function further generated `char_ids`, `start_ids`, and `end_ids` to record the character sequence and its position alignment under the same maximum length.

For the lexical analysis feature branch, each URL was parsed into domain, path, filename, extension, and query components using `urlparse`. We extracted 79 handcrafted URL features following Mamun et al. [6],

Table 1

Comparison of malicious URL detection studies reviewed in Sections 2.2–2.4.

Category	Model Architecture	Dataset used	Feature Type	Evaluation metrics	Advantages	Limitations
Lexical and traditional ML	Descriptive phishing modus operandi analysis [18]	Two-month phishing URL feeds from PhishTank and MarkMonitor; WHOIS, zone files, DNS resolutions; DMOZ and older phishing URLs for comparison	Lexical + Host-based features	Descriptive statistics	Quantified URL/domain, registration, and hosting patterns that motivated lexical heuristics	No supervised detector or accuracy/F1; limited direct adaptation to evolving attacks
	LR, SVM, Naive Bayes, and decision tree with Grid Search tuning [23]	Kaggle malicious URL classification dataset with 420,464 URLs; balanced benign/malicious classes of 210,232 URLs each	Lexical + Host-based features	Accuracy, precision, recall, F1-score	Provides a recent practical benchmark of classical ML classifiers with deployable web-service implementation	Depends on engineered URL features and a single Kaggle dataset; no cross-dataset, adversarial, or concept-drift evaluation
	Naive Bayes, SVM, and logistic regression with high-dimensional feature selection [20]	DMOZ/Yahoo benign URLs paired with PhishTank or Spamscatter malicious URLs; DP, DS, YP, and YS sets of about 20K–30K URLs	Lexical + Host-based features	Accuracy, false-positive rate	Moves beyond blacklists using URL and host metadata without page download	Depends on public host metadata and feature distribution; some evasion cases require content analysis
	Random Forest with TF-IDF and hand-crafted URL features [21]	CommonCrawl legitimate URLs (42,220), Alexa legitimate URLs (43,189), PhishTank phishing URLs (40,668), plus benchmark D4/D5 datasets	Lexical Features	Accuracy, precision, F-measure	URL-only, fast, no third-party lookup or website visit; avoids drive-by-download exposure	May miss visual mimicry, shortened/data URLs, and attacks weakly expressed in URL strings
	RF and six other ML classifiers with NLP, word-vector, and hybrid URL features [22]	Ebbu2017 dataset with 73,575 URLs: 36,400 legitimate URLs collected via Yandex and 37,175 phishing URLs mainly from PhishTank	Lexical Features	Accuracy, precision, sensitivity, F-measure	Real-time, language-independent, third-party-independent URL detector with a relatively large public dataset	Still feature-engineered; weaker for short/domain-only URLs and may need extra active features
CNN-based malicious URL detection	Parallel CNN-CapsNet with convolutional and capsule layers [23]	4,820,940 URLs: 241,047 phishing from PhishTank/OpenPhish and 4,579,893 legitimate URLs crawled from Alexa top sites	Character-level features	Accuracy, precision, recall, F1-score, AUC	Captures shallow and hierarchical URL patterns in an imbalanced real-world-like setting	URL-only model; fixed supervised training still requires updates for new attack distributions
	CBIR hybrid CNN-BiLSTM-IndRNN architecture [24]	PhishTank phishing URLs (13,652) and crawler-collected legitimate URLs (10,000)	Character + word-based + Lexical features	Accuracy	Fuses texture, URL word-vector, host, and URL static vocabulary information	Complex feature extraction and model pipeline; precision/recall/F1 are not explicitly reported
	URLNet dual-branch character- and word-level CNN [7]	VirusTotal-crawled large-scale corpus: 5M training URLs and 10M testing URLs; 15M total with 949,725 malicious URLs	Character + word-based features	AUC, TPR, FPR	Learns multi-granularity URL representations without manual feature engineering	Requires very large labeled data; fixed learned representations may lag concept drift
	GramBeddings four-channel CNN-BiLSTM-attention model [25]	GramBeddings dataset with 800K URLs: 400K phishing URLs from PhishTank/OpenPhish and 400K legitimate URLs crawled from Alexa/top sites; additional benchmark datasets	Character-level features	Accuracy, precision, recall, F1-score, AUC	End-to-end, language-agnostic, no hand-crafted or third-party features; includes adversarial robustness study	More complex neural architecture; adversarial robustness improves only after adversarial retraining
Transformer-based malicious URL detection	Fine-tuned BERT malicious URL classifier [8]	105,930 URLs: 27,890 phishing, 36,700 SQL injection, 21,340 sensitive-file, and 20,000 normal URLs from Alexa	Wordpiece-level features	Accuracy, recall, F1-score	Introduces contextual PLM representations for URL classification	General BERT transfer is not URL-specific and may suffer domain shift; dataset is multi-category rather than only benign/malicious
	URLTran BERT/roBERTa Transformer with BPE and MLM pretraining [26]	Microsoft Edge/Internet Explorer production telemetry: 1,039,413 training, 259,854 validation, and 1,784,155 testing URLs	Wordpiece-level features	Accuracy, precision, F1-score, AUC, TPR, FPR	Production-scale evaluation and low-FPR reporting; includes adversarial perturbation analysis	Relies on proprietary telemetry and labels; fine-tuned model still needs updating for new attacks

(continued on next page)

Table 1 (continued)

Category	Model Architecture	Dataset used	Feature Type	Evaluation metrics	Advantages	Limitations
	Adversarially post-trained BERT with multi-view MoE Transformer [27]	PhishTank URLs before Nov. 2022 (2,198,157) and 40,066 benign URLs from top-ranked websites; balanced and imbalanced evaluation subsets	Wordpiece-level + Lexical + Context features	Accuracy, TPR, FPR, AUC, F1-score	Multi-view fusion improves robustness under imbalance, small-label settings, and data drift	Requires website attribute/content/behavior collection, increasing deployment overhead beyond URL-only methods
	Lightweight character-level Transformer classifier [28]	20,000 URLs: 10,000 benign URLs from UNB ISCX-URL2016 and 10,000 malicious URLs from PhishTank; 80/20 train-test split	Character-level features	Accuracy, precision, recall, F1-score	Simple URL-only Transformer outperforming six classical baselines in the reported comparison	Small balanced dataset; authors note larger training data is needed for stronger robustness
	TransURL CharBERT-based multi-layer Transformer with multi-scale pyramid attention [11]	GramBeddings, Mendeley, Kaggle 1, and Kaggle 2 public URL datasets for binary, imbalanced, cross-dataset, multi-class, and adversarial tests	Character + word-based features	Accuracy, precision, recall, F1-score, AUC	Captures character-aware, local, multi-scale, and global URL patterns; robust across challenging scenarios	Heavier than plain URL-only classifiers and depends on a pretrained CharBERT backbone
	CharBERT dual-channel character-aware PLM with BiGRU character encoder [13]	English Wikipedia pretraining corpus (12G, 2.5B words); evaluated on SQuAD, GLUE-style classification, CoNLL-2003 NER, POS, and misspelling attacks	Character + word-based features	SQuAD EM/F1, classification accuracy, NER/POS F1, and robustness metrics; no URL-detection metrics	Provides character-aware representations useful for obfuscated strings and later URL models such as TransURL	Not a malicious URL detector by itself; requires costly PLM pretraining and URL-domain adaptation

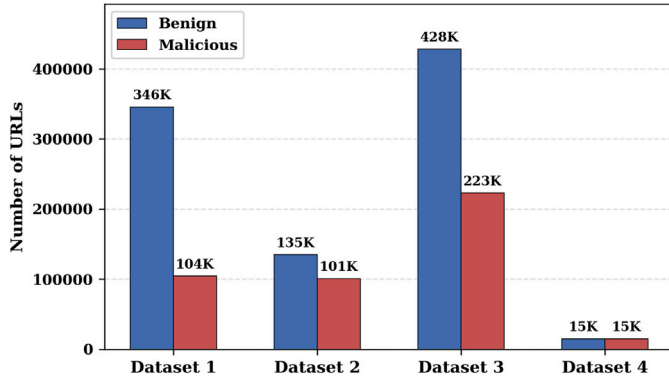


Fig. 2. The number of malicious and benign samples in each dataset.

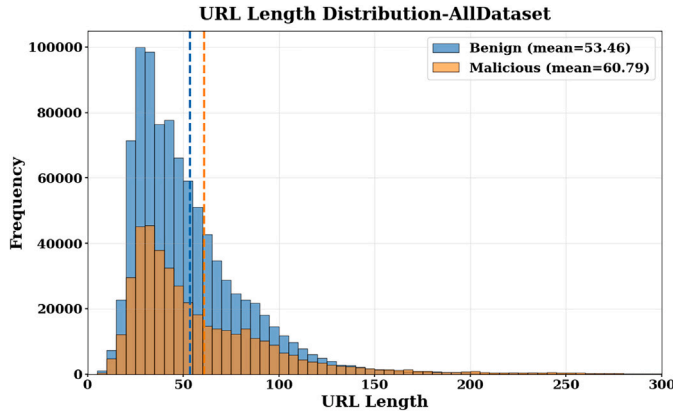


Fig. 3. Distribution chart of URL lengths for all datasets.

including length statistics, token counts, digit and letter counts, symbol counts, delimiter counts, ratio features, sensitive word indicators, character continuity measures, and entropy values. These features were subsequently used for saliency-based feature selection.

The preprocessing strategy was intentionally conservative to preserve URL surface patterns that may contain security signals, including digits, delimiters, path fragments, query strings, and symbolic structures. Because the public benchmark datasets used in this study were released in processed form, we treated the provided records as the reference data

and avoided additional duplicate removal that could alter the benchmark distribution. We also avoided aggressive noisy-sample filtering, since short, irregular, or symbol-rich URL strings may themselves be informative for malicious URL detection. Robustness was instead maintained during input construction through fixed-length truncation and padding, printable-character filtering, unknown-character fallback, and zero-value fallback for exceptional handcrafted feature extraction cases.

4.2. Grammar features after dual-channel fusion

As illustrated in Fig. 6, we use CharBERT as the backbone encoder of DCG-BERT. Standard BERT represents an input mainly through token-level embeddings. This design may overlook character-level obfuscations that often appear in malicious URLs, such as substituted letters or abnormal spelling patterns. CharBERT addresses this limitation by maintaining two parallel channels: a Token Channel and a Character Channel. A layer-wise Heterogeneous Interaction (HI) module then exchanges information between the two channels.

4.2.1. Dual-channel construction

Given an input URL, we first tokenize it into a subword sequence $w = \{w_1, w_2, \dots, w_m\}$ using the WordPiece algorithm.

Token Channel: The token channel follows the standard BERT embedding scheme. For the i -th subword w_i at position pos_i , the initial token representation is computed by summing the word, position, and token-type embeddings, followed by layer normalization:

$$t_i^{(0)} = \text{LayerNorm}(\text{WordEmb}(w_i) + \text{PosEmb}(pos_i) + \text{TypeEmb}) \quad (1)$$

This gives the initial token representation $T^{(0)} = \{t_1^{(0)}, \dots, t_m^{(0)}\}$.

Character Channel (CM Module): The character channel captures the internal morphology of each subword. As shown in Fig. 7, the Character Module (CM) uses a Bidirectional GRU (BiGRU). Let the subword w_i consist of a character sequence $\{c_1^i, \dots, c_{n_i}^i\}$. These characters are first mapped to embeddings and then processed in both directions:

$$\vec{h}_j = \text{GRU}_{fwd}(\vec{h}_{j-1}, c_j) \quad (2)$$

$$\overleftarrow{h}_j = \text{GRU}_{bwd}(\overleftarrow{h}_{j+1}, c_j) \quad (3)$$

$$h_j = [\vec{h}_j; \overleftarrow{h}_j] \quad (4)$$

where $[\cdot; \cdot]$ denotes concatenation.

To represent the whole subword, we concatenate the hidden states of its first and last characters:

$$h_i^{(0)}(x) = [h_1^i(x); h_{n_i}^i(x)] \quad (5)$$

This representation preserves both boundary information and internal character morphology.

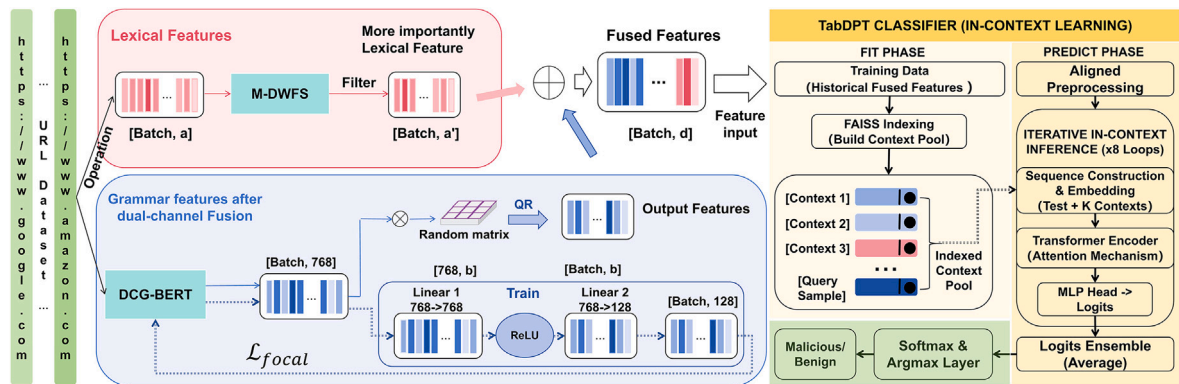


Fig. 4. Overall workflow of the proposed MLF-ICL framework. DCG-BERT extracts deep grammar features from raw URL strings through a dual-channel architecture and context gating. M-DWFS selects discriminative lexical features through dynamic weighting. TabDPT performs the final classification by using retrieval-augmented In-Context Learning. The symbols $\oplus$ and $\otimes$ denote concatenation and matrix multiplication, respectively.

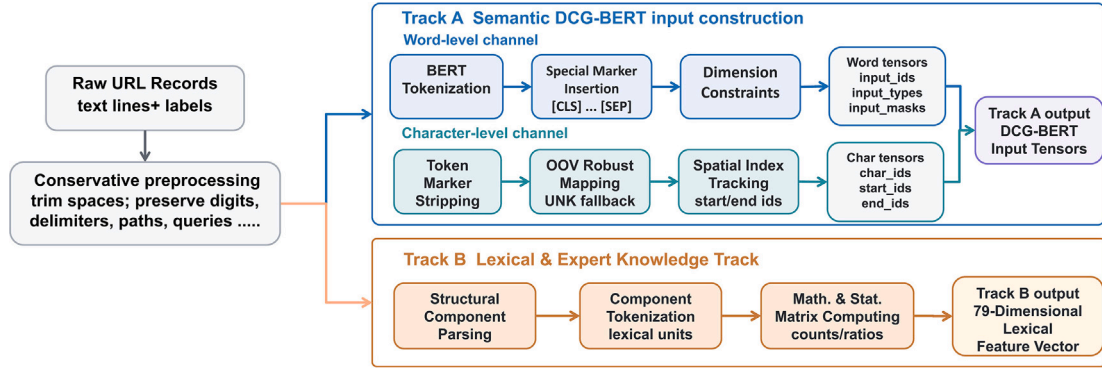


Fig. 5. Preprocessing workflow for URL feature construction, including paired word and character inputs for CharBERT and 79 handcrafted lexical analysis features extracted from parsed URL components.

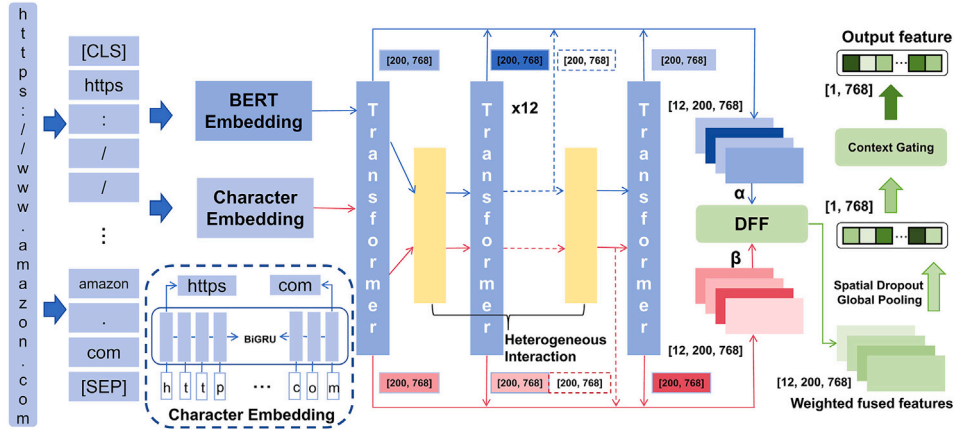


Fig. 6. Workflow of the DCG-BERT module for grammar feature extraction. The HI module integrates the token and character channels after each Transformer layer. The DFF module then adaptively fuses and stacks the outputs from all 12 CharBERT layers. Finally, Context Gating recalibrates the global representation to emphasize discriminative information.

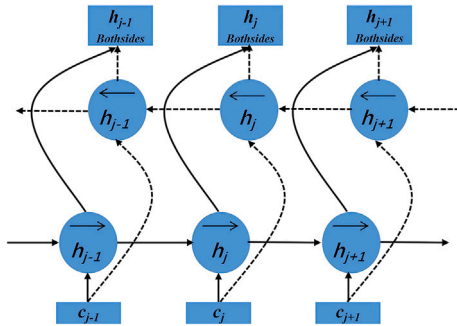


Fig. 7. Character encoder module.

4.2.2. Heterogeneous interaction (HI) module

While independent token and character channels can respectively capture macroscopic semantics and microscopic morphology, maintaining them in isolation throughout the network depth prevents the model from understanding their correlation. Malicious URLs frequently employ subtle morphological obfuscations (e.g., substituting ‘o’ with ‘0’ in ‘google’) to alter character composition while attempting to preserve the visual token semantics. Therefore, an explicit interaction mechanism is required. The HI module is designed to bridge this semantic gap. It acts as a dynamic conduit that forces character-level anomalies

to continually recalibrate token-level semantic representations at each Transformer layer.

To address this problem, we insert an HI module into each of the 12 Transformer layers, as shown in Fig. 8. For the l -th layer, the HI module follows a Fusion-Divide-Update process.

Fusion Stage: The token and character representations from the previous layer, $T^{(l-1)}$ and $H^{(l-1)}$, are first projected into a shared feature space:

$$t'^{(l)}(x) = W_1 T^{(l-1)}(x) + b_1 \quad (6)$$

$$h'^{(l)}(x) = W_2 H^{(l-1)}(x) + b_2 \quad (7)$$

where W_1 , W_2 , b_1 , and b_2 are learnable parameters.

The projected features are concatenated as $w^{(l)}(x) = [t'^{(l)}(x); h'^{(l)}(x)]$. A layer-specific 1D-CNN fusion module is then applied to the concatenated sequence to capture local dependencies between token semantics and character morphology, enabling each Transformer layer to learn its own token-character interaction pattern:

$$m_j^{(l)}(x) = \tanh(W_3^{(l)} \cdot w_{j:j+s-1}^{(l)}(x) + b_3^{(l)}) \quad (8)$$

where $w_{j:j+s-1}^{(l)}$ denotes the local context window at position j , s is the window size, and $W_3^{(l)}$ and $b_3^{(l)}$ are the layer-specific convolutional parameters.

Divide & Update Stage: The fused representation $m^{(l)}(x)$ is then mapped back into the two channels. Two independent, fully connected

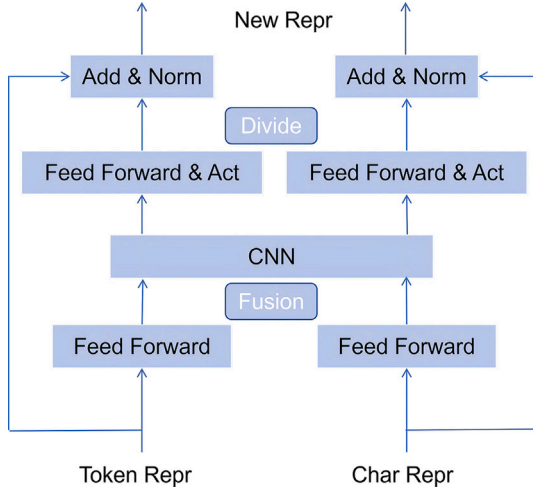


Fig. 8. Heterogeneous Interaction (HI) module.

layers generate channel-specific update vectors:

$$m^{(l)}(x) = \Delta(W_4 m^{(l)}(x) + b_4) \quad (9)$$

$$m^{h(l)}(x) = \Delta(W_5 m^{(l)}(x) + b_5) \quad (10)$$

where Δ denotes the GELU activation function.

The update vectors are added to the previous token and character representations through residual connections. Layer normalization is then applied to stabilize training:

$$T^{(l)}(x) = \text{LayerNorm}(T^{(l-1)}(x) + m^{(l)}(x)) \quad (11)$$

$$H^{(l)}(x) = \text{LayerNorm}(H^{(l-1)}(x) + m^{h(l)}(x)) \quad (12)$$

Liu et al. [12] showed that using embeddings from all 12 layers improves performance. Following this observation, CharBERT produces the interactive representations $\{T^{(l)}, H^{(l)}\}_{l=1}^{12}$. These representations are then passed to the Dynamic Fusion Framework.

4.2.3. Dynamic fusion framework (DFF)

Different Transformer layers encode different types of information. Lower layers tend to capture local or phrase-level patterns, whereas higher layers focus on more abstract semantics [39,40]. Relying only on the final layer may discard useful low-level morphological cues. This is especially problematic for malicious URL detection, where small character changes can be discriminative.

DFF therefore fuses representations from all 12 layers. For each layer l , the token representation $T^{(l)}$ and character representation $H^{(l)}$ are concatenated and averaged to form a global context vector. A lightweight weight-generation network then computes the normalized fusion weights:

$$(\alpha^{(l)}, \beta^{(l)}) = \text{Softmax}(\text{WeightGen}(\text{Mean}([T^{(l)}, H^{(l)}]))) \quad (13)$$

The fused representation $F^{(l)}$ is obtained by adaptive weighted summation followed by layer normalization. The fused outputs from all layers are then stacked into a multi-order tensor $P \in \mathbb{R}^{B \times L \times S \times D}$.

Spatial Dropout Regularization: Before the final reshaping step, we apply Spatial Dropout (Dropout1d) to the stacked tensor. This operation drops entire feature channels across sequence positions. It reduces inter-channel dependence and helps prevent memorization of training-specific patterns [41].

4.2.4. Context gating & feature recalibration

The multi-order tensor $P_{\text{final}} \in \mathbb{R}^{B \times L \times S \times D}$ still contains redundant and noisy information. We therefore introduce a Context Gating module to recalibrate the global representation, as shown in Fig. 9.

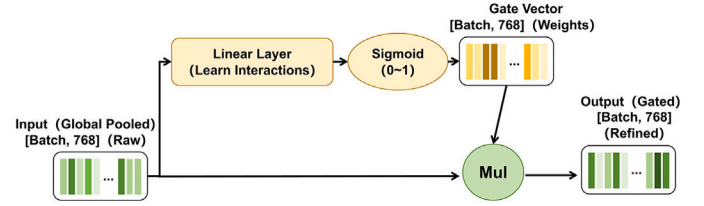


Fig. 9. Context Gating Module.

Global Pooling: We first compress the tensor along the layer dimension (L) and sequence dimension (S). This produces a global feature vector in $\mathbb{R}^{B \times D}$.

Gating Coefficient Computation: A linear projection followed by a sigmoid activation generates a gating vector: $\text{gates} \in [0, 1]^D$. The gating vector models the importance of each feature dimension.

Feature Recalibration: The global feature vector is reweighted by element-wise multiplication with the gating vector:

$$\text{gated_features} = \text{global_features} \odot \sigma(W \cdot \text{global_features} + b) \quad (14)$$

where $\text{gated_features} \in \mathbb{R}^{B \times D}$. This step suppresses redundant dimensions and emphasizes discriminative URL patterns.

Finally, we use Orthogonal Random Projection to compress the recalibrated vector for downstream tabular inference. We first construct a random matrix $A \in \mathbb{R}^{D \times b}$, where b is the target dimension. The entries of A are sampled from a standard normal distribution. QR decomposition is then used to obtain an orthogonal basis matrix R . The compact grammar feature is computed as

$$u = \text{gated_features} \cdot R \in \mathbb{R}^{B \times b} \quad (15)$$

This projection reduces feature dimensionality while approximately preserving pairwise Euclidean distances, following the Johnson-Lindenstrauss lemma.

4.2.5. Training objective and dataset imbalance handling

The orthogonal projection in Eq. (15) is used to obtain the compact grammar feature u_i for downstream tabular inference. However, during representation learning, the supervised training objective is applied before this projection. Specifically, the auxiliary classification head is attached directly to the gated representation $g_i \in \mathbb{R}^d$ produced by the Context Gating module. This design is consistent with the implementation, where the classification head uses the recalibrated feature vector rather than the projected downstream feature. The logits for sample i are computed as

$$o_i = W_c g_i + b_c, \quad (16)$$

where $W_c \in \mathbb{R}^{C \times d}$ and $b_c \in \mathbb{R}^C$ denote the classifier parameters, and $C = 2$ is the number of target classes. For notational simplicity, the dropout operation applied before the classifier is omitted in Eq. (16). The predicted probability for class c is obtained by the softmax function:

$$\hat{p}_{i,c} = \frac{\exp(o_{i,c})}{\sum_{j=1}^C \exp(o_{i,j})}. \quad (17)$$

To address dataset imbalance, we adopt a combined strategy at the sampling and loss levels. First, for the imbalanced training set, we use weighted random sampling. The sampling weight of class k is computed only from the training set as

$$\omega_k^{\text{sam}} = \frac{1}{n_k^{\text{train}} + \epsilon}, \quad (18)$$

where n_k^{train} denotes the number of training samples in class k , and $\epsilon = 10^{-6}$ is used for numerical stability. Each sample inherits the weight of

its class, so minority-class samples have a higher probability of being drawn into each mini-batch.

Second, we use focal loss to reduce the dominance of easy majority-class samples. For sample i , the cross-entropy loss is

$$\mathcal{L}_i^{CE} = -\log \frac{\exp(o_{i,y_i})}{\sum_{c=1}^C \exp(o_{i,c})}, \quad (19)$$

and the corresponding focal loss is

$$\mathcal{L}_i^{FL} = (1 - \hat{p}_{i,y_i})^{\gamma_f} \mathcal{L}_i^{CE}, \quad (20)$$

where $\gamma_f = 2.0$. No additional class coefficient is used because class imbalance is explicitly handled by the sampler.

Third, hard negative mining is applied within each mini-batch. Let B_1 and B_0 denote the malicious and benign samples in a mini-batch B , respectively. All malicious samples are retained, while only the highest-loss benign samples are selected:

$$\mathcal{H}_0 = \text{TopK}_{|r_h|B_0|} \{i \in B_0; \mathcal{L}_i^{FL}\}, \quad (21)$$

where $r_h = 0.5$ is the hard-negative retention ratio.

Ultimately, the final training objective for optimizing the DCG-BERT representation-learning module and its auxiliary classification head is formulated as the average focal loss over the curated mini-batch:

$$\mathcal{L}_{total} = \frac{1}{|B_1| + |\mathcal{H}_0|} \sum_{i \in B_1 \cup \mathcal{H}_0} \mathcal{L}_i^{FL}. \quad (22)$$

After training, the classification head is used solely to guide representation learning. The compact grammar feature u_i obtained from Eq. (15) is subsequently concatenated with the lexical feature vector generated in Section 4.2 to form the final tabular representation for TabDPT-based inference.

4.3. Lexical analysis features

Deep neural representations capture latent semantic patterns in URLs. However, explicit lexical cues remain useful for identifying phishing artifacts. These cues include abnormal URL length, suspicious keywords, digit-letter mixtures, special symbols, and high-entropy strings.

We therefore construct a lexical feature pathway in parallel with the grammar feature pathway. First, we extract 79 handcrafted URL features following Mamun et al. [6]. These features describe morphological, statistical, heuristic, and information-theoretic properties of the URL. Second, we apply MLP Saliency Dynamic Weighted Feature Selection (M-DWFS) to remove redundant features and retain the most discriminative subset. M-DWFS uses the Top-50 lexical features identified by Naseer et al. [9] as a prior feature list, and then adjusts their weights according to the current dataset.

4.3.1. Multi-dimensional feature generation

The raw URL is first parsed into its domain, path, and query components. Based on these components, we extract an a -dimensional lexical feature vector $n \in \mathbb{R}^a$. The extracted features cover four categories:

1. **Morphological Features:** These features describe the structural form of a URL. Examples include URL length, domain length, path depth, token count, and token length statistics.
2. **Statistical Features:** These features measure character-level distributions. They include the counts and ratios of digits, letters, and special symbols in different URL components.
3. **Heuristic Indicators:** These features encode security-domain rules. Examples include IP-based domains, executable file extensions, port indicators, and sensitive keywords such as “login” and “bank”.
4. **Information-Theoretic Features:** These features measure the randomness of URL strings. They are useful for detecting algorithmically generated domains or obfuscated URL components. For a URL string V , the Shannon entropy is computed as

$$H_{\text{lexical}}(V) = -\sum_{i=1}^n p(v_i) \log_2 p(v_i) \quad (23)$$

where v_i denotes a unique character and $p(v_i)$ denotes its frequency in V (Table 2).

4.3.2. MLP saliency dynamic weighted feature selection (M-DWFS)

The 79 lexical features contain useful information, but some features are redundant or weakly related to the target label. To reduce this redundancy, we propose M-DWFS. As summarized in Algorithm 1, M-DWFS builds on the idea of dynamic weighted feature selection [42] by combining gradient-based MLP saliency with information-theoretic redundancy analysis. It also uses the Top-50 prior feature list identified by Naseer et al. [9] to initialize feature weights.

M-DWFS consists of three steps.

1. **Saliency-based Initialization:** We first standardize the lexical feature matrix N into N' using Z-score normalization. A lightweight multilayer perceptron (MLP) is then trained on N' with binary cross-entropy loss L . After training, the saliency score of each feature is computed from the gradient of the loss with respect to the input:

$$s_i = \frac{1}{M} \sum_{j=1}^M \left| \frac{\partial L}{\partial n_i^{(j)}} \right| \quad (24)$$

where M is the number of samples used for saliency estimation.

Table 2

Categories of the 79 extracted URL lexical features. Features in **bold** belong to the Top-50 prior feature list used by M-DWFS.

Feature Category	Features
Morphological Features	domainlength , urlLen , tld , path_token_count , domain_token_count , longdomaintokenlen , avgdomaintokenlen , pathurlRatio , domainUrlRatio , pathDomainRatio , argPathRatio , ArgUrlRatio , LongestPathTokenLength , Domain_LongestWordLength , Arguments_LongestWordLength , sub-Directory_LongestWordLength , delimiter_Domain , delimiter_Count , this.fileExtLen , Querylength , avgpathtokenlen , pathLength , subDirLen , fileNameLen , ArgLen , argDomanRatio , Path_LongestWordLength , delimiter_path
Statistical Features	Directory_DigitCount , File_name_DigitCount , URL_DigitCount , Directory_LetterCount , URL_Letter_Count , Extension_LetterCount , Query_LetterCount , host_letter_count , SymbolCount_Extension , SymbolCount_FileName , SymbolCount_Afterpath , SymbolCount_Domain , SymbolCount_URL , NumberRate_Domain , NumberRate_URL , NumberRate_AfterPath , charcompvowels , charcompce , CharacterContinuityRate , host_DigitCount , Extension_DigitCount , Query_DigitCount , Filename_LetterCount , NumberRate_DirectoryName , NumberRate_FileName , NumberRate_Extension , SymbolCount_Directoryname
Heuristic Indicators	spcharUrl , isPortEighty , ldl_getArg , ldl_domain , ldl_filename , dld_domain , dld_filename , dld_url , NumberofDotsinURL , executable , ISIpAddressInDomainName , LongestVariableValue , URL_sensitiveWord , URLQueries_variable , ldl_url , ldl_path , dld_path , dld_getArg
Information-Theoretic	Entropy_Afterpath , Entropy_URL , Entropy_Domain , Entropy_DirectoryName , Entropy_Filename , Entropy_Extension

Algorithm 1 M-DWFS Algorithm.

```

1: Input: Dataset  $D$ , prior feature list  $P$  (Top-50)
2: Output: Selected feature subset  $S$ 
3: Initialization:
   Continuous features  $X_{\text{cont}}$ ; discretized features  $X_{\text{disc}}$ ; labels  $Y$ ;
   candidate feature set  $F$ ; target size  $d$ ; saliency score  $R$ ;
   feature weight  $W$ ; selected subset  $S \leftarrow \emptyset$ 
4:  $X_{\text{cont}}, Y \leftarrow \text{Split\_and\_Scale}(D)$ 
5:  $X_{\text{disc}} \leftarrow \text{Discretize}(X_{\text{cont}})$ 
6: // Phase 1: MLP saliency calculation
7:  $\text{Model} \leftarrow \text{Train\_MLP}(X_{\text{cont}}, Y)$ 
8:  $R \leftarrow \text{Compute\_Gradient\_Saliency}(\text{Model}, X_{\text{cont}})$ 
9: // Phase 2: adaptive weight initialization
10:  $\text{factor} \leftarrow \text{Calc\_Adaptive\_Factor}(R, P)$ 
11:  $W \leftarrow \text{Initialize\_Weights}(1.0)$ 
12:  $W[P] \leftarrow \text{factor}$ 
13: // Phase 3: dynamic weighted feature selection
14: while  $|S| < d$  do
15:    $J \leftarrow R \odot W$ 
16:    $f_{\text{best}} \leftarrow \text{argmax}(J)$ 
17:    $S \leftarrow \text{Append}(S, f_{\text{best}})$ 
18:    $F \leftarrow \text{Remove}(F, f_{\text{best}})$ 
19:   for each  $f_i$  in  $F$  do
20:      $CR \leftarrow \text{Correlation\_Ratio}(f_i, f_{\text{best}}, X_{\text{disc}}, Y)$ 
21:      $W[f_i] \leftarrow W[f_i] \times (1 + CR)$ 
22:   end for
23: end while
24: return  $S$ 

```

2. **Data-driven Dynamic Weighting:** We use an adaptive factor w_{dyn} to weight the prior feature set P , where $|P| = 50$. The factor depends on the relative saliency of the prior features and the dispersion of saliency scores:

$$R_{\text{ratio}} = \frac{\mu_P}{\mu_A} \quad (25)$$

$$D_{\text{adj}} = \frac{1.0}{1 + \sigma_A / \sigma_P} \quad (26)$$

where μ and σ denote the mean and standard deviation of saliency scores. The subscripts P and A refer to the prior feature set and all features, respectively. The final dynamic weight is

$$w_{\text{dyn}} = 1.0 + (R_{\text{ratio}} - 1.0) \cdot D_{\text{adj}} \cdot \alpha_{\text{scale}} \cdot \beta_{\text{balance}} \quad (27)$$

where $\alpha_{\text{scale}} = \min(1.0, N_{\text{total}}/10000)$ adjusts for sample size, and β_{balance} adjusts for class imbalance. Features in P are initialized with w_{dyn} , and the remaining features are initialized with weight of 1.0.

3. **Iterative Selection with CMI-based Update:** The algorithm iteratively selects a subset S of size a' . At each iteration, it selects the remaining feature with the largest weighted saliency score:

$$j = \arg \max_{i \in F} (s_i \cdot w_i); \quad S \leftarrow S \cup \{j\} \quad (28)$$

After selecting feature j , M-DWFS updates the weights of the remaining features according to their redundancy or complementarity with j . We compute a correlation ratio using mutual information (MI) and conditional mutual information (CMI):

$$CR_{kj} = \frac{2 \cdot (CMI(k, y|j) - MI(k, y))}{H_{\text{lexical}}(k) + H_{\text{lexical}}(y) + \epsilon} \quad (29)$$

$$MI(k, y) = H_{\text{lexical}}(k) + H_{\text{lexical}}(y) - H_{\text{lexical}}(k, y) \quad (30)$$

$$CMI(k, y|j) = H_{\text{lexical}}(k, j) + H_{\text{lexical}}(y, j) - H_{\text{lexical}}(k, y, j) - H_{\text{lexical}}(j) \quad (31)$$

where y is the label and ϵ prevents division by zero. The feature weight is then updated as $w_k \leftarrow w_k \cdot (1 + CR_{kj})$. This rule decreases the weight of redundant features when $CR_{kj} < 0$ and increases the weight of complementary features when $CR_{kj} > 0$.

The final output of M-DWFS is the selected lexical feature vector $m \in \mathbb{R}^{d'}$, which is later concatenated with the grammar feature vector for TabDPT-based inference.

4.4. TabDPT classification framework

After feature construction, each URL is represented as a multimodal tabular vector. The vector is formed by concatenating the compact grammar feature u_i from Section 4.1 and the selected lexical feature vector m_i from Section 4.2:

$$x_i = [u_i; m_i] \in \mathbb{R}^{d_x}, \quad (32)$$

where $d_x = b + a'$ is the input feature dimension. TabDPT is then used as the final inference module. Unlike conventional classifiers that learn fixed decision boundaries through task-specific fine-tuning, TabDPT performs retrieval-augmented In-Context Learning (ICL). It predicts the label of a test sample by comparing it with relevant historical examples, without updating model parameters during inference.

4.4.1. Non-parametric fitting and indexing

The fit phase organizes the training data for retrieval. It does not optimize the parameters of the pre-trained TabDPT model. Given the training set $D_{\text{train}} = \{(x_i, y_i)\}_{i=1}^n$, we first preprocess the feature vectors and then build a retrieval index.

1. **Imputation and Scaling:** Missing values are imputed using the column-wise mean μ . The imputed features are then standardized by the column-wise standard deviation σ :

$$x'_i = \frac{\text{Impute}(x_i, \mu) - \mu}{\sigma} \quad (33)$$

where μ and σ are computed only from the training set.

2. **Context Indexing:** We build a FAISS index I over the processed training vectors X_{proc} . Specifically, we use an IndexFlatL2 structure, which retrieves neighbors according to Euclidean distance. The index serves as external memory for context retrieval during inference.

4.4.2. Context-aware inference with feature permutation

For a test URL x_i , the prediction phase retrieves relevant context samples and feeds them into TabDPT. To reduce sensitivity to feature order, we use a feature permutation ensemble. The following process is repeated $N = 8$ times with different random seeds.

1. **Context Retrieval:** The test sample is preprocessed using the stored training statistics μ and σ to obtain x'_i . We then query the index I and retrieve the top- k nearest neighbors, with $k = 2048$. The retrieved context set is denoted as $C = \{(x'_{c_j}, y_{c_j})\}_{j=1}^k$.
2. **Feature Permutation:** For the n -th ensemble run, we generate a random permutation matrix $P_n \in \mathbb{R}^{d_x \times d_x}$. The same permutation is applied to both the context vectors and the test vector:

$$x'_{\text{perm}} = x' P_n. \quad (34)$$

This provides different feature-order views of the same sample.

3. **Sequence Construction and Embedding:** We construct a sequence of k context examples followed by the test query. For each

context example, the embedding combines feature information and label information:

$$z_j = \text{Linear}_x(x'_{c_j, \text{perm}}) + \text{Linear}_y(y_{c_j}), \quad j = 1, \dots, k \quad (35)$$

For the test query, the label is masked and only the feature embedding is used:

$$z_{k+1} = \text{Linear}_x(x'_{t, \text{perm}}) \quad (36)$$

The final input sequence is $Z = [z_1, \dots, z_k, z_{k+1}] \in \mathbb{R}^{(k+1) \times d_e}$, where d_e is the Transformer embedding dimension.

4.4.3. Deep in-context reasoning

The sequence Z is processed by a stack of L Transformer encoder layers and a classification head. Before entering the encoder, Z is normalized by RMS normalization to stabilize the feature distribution.

In each encoder layer, TabDPT uses an asymmetric multi-head self-attention mechanism. The query projection Q is computed from the full sequence, including both context examples and the test sample. The key and value projections, K and V , are computed only from the k context examples. This design forces the retrieved examples to act as the reference context.

The attention output is computed as

$$\text{Attn} = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_{\text{head}}}} \right) V \in \mathbb{R}^{(k+1) \times d_e} \quad (37)$$

where d_{head} is the dimension of each attention head. The output is then processed by a feed-forward network with residual connections. The final encoder output is denoted as $H_{\text{last}} \in \mathbb{R}^{(k+1) \times d_e}$.

For classification, we extract the last vector from H_{last} , which corresponds to the test sample. Let this vector be h . A two-layer classification head maps h to class logits:

$$h_{\text{mid}} = \text{GELU}(hW_1 + b_1) \in \mathbb{R}^{d_{\text{mid}}} \quad (38)$$

$$\text{logits} = h_{\text{mid}}W_2 + b_2 \in \mathbb{R}^C \quad (39)$$

where C is the number of target classes.

Finally, the logits from the $N = 8$ permutation runs are averaged. The averaged logits are converted into class probabilities using a temperature-scaled softmax:

$$p(x_t) = \text{Softmax} \left(\frac{1}{\tau} \cdot \frac{1}{N} \sum_{n=1}^N \text{logits}_n \right) \in \mathbb{R}^C \quad (40)$$

where τ is the temperature coefficient. The final predicted label is the class with the highest probability.

5. Experiment analysis

This section provides a systematic overview of the proposed methods, detailed parameter configurations, and evaluation metrics utilized in this study. To align with our framework, we adopted a rigorous and sequential dataset partitioning strategy. First, to train the feature extraction backbone, each dataset was randomly partitioned into a 72:8:20 ratio for training, validation, and testing, respectively. Second, for the TabDPT classification stage, the dataset allocation transitioned into an 80/20 split. Specifically, the 80% reference set used to construct the FAISS index was formed by combining the training and validation sets from the first stage. The remaining 20% served as the strictly unseen testing set to evaluate the final classification performance. All the evaluation experimental results in this chapter are the average values and standard deviations obtained through 5-fold cross-validation.

5.1. Evaluation metrics

In this section, we employ Accuracy, Precision, Recall, False Positive Rate (FPR), and F1-score as the primary performance metrics. The fundamental components of the confusion matrix are defined as follows: TP, TN, FP, and FN represent True Positive, True Negative, False Positive, and False Negative, respectively. The definitions of the specific metrics are presented below:

Accuracy: Accuracy is the most intuitive metric for measuring the overall correctness of the model's classification. It calculates the proportion of correctly classified instances (both TP and TN) relative to the total number of samples. This reflects the model's comprehensive predictive capability across the entire dataset, as shown in Eq. (41).

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (41)$$

Precision: Also referred to as Positive Predictive Value (PPV), Precision focuses on the "exactness" of the model's predictions. It quantifies the proportion of instances that are actually positive among all instances predicted as positive by the model, as shown in Eq. (42)

$$\text{Precision} = \frac{TP}{TP + FP} \quad (42)$$

Recall: Also known as Sensitivity or True Positive Rate (TPR), Recall focuses on the "completeness" of the model's predictions. It quantifies the fraction of actual positive instances that are successfully identified by the model, as shown in Eq. (43)

$$\text{Recall} = \frac{TP}{TP + FN} \quad (43)$$

F1-Score: The F1-Score is the harmonic mean of Precision and Recall. It is designed to provide a balanced metric, particularly in scenarios involving class imbalance or when Precision and Recall are of equal importance, as shown in Eq. (44).

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (44)$$

Specificity: Also known as True Negative Rate (TNR), Specificity focuses on the "completeness" of the model's negative predictions. It quantifies the fraction of actual negative instances that are successfully identified by the model, as shown in Eq. (45).

$$\text{Specificity} = \frac{TN}{TN + FP} \quad (45)$$

AUROC & AUPRC: AUROC and AUPRC quantify the area under the ROC and PR curves, respectively. The ROC curve plots the True Positive Rate against the False Positive Rate to show classification capability, while the PR curve plots Precision against Recall at various thresholds, which is especially effective for imbalanced datasets.

5.2. Parameter settings

For the training of the backbone model, the optimizer was configured with an initial learning rate of 2×10^{-5} and a weight decay of 1×10^{-4} . The training process spanned 3 epochs with a batch size of 32.

Regarding the handcrafted feature sub-network, the architecture consists of two hidden layers with 128 and 64 units, respectively. We employed the Adam optimizer with a learning rate of 0.001 and a batch size of 32. This module was trained for 10 epochs with a specific weight factor of 1.5.

For the TabDPT classifier, the maximum number of features F_{max} was fixed at 100. The key hyperparameters were set as follows: number of ensembles $n_{\text{ensembles}} = 8$, temperature = 0.3, and context size = 2048. Consequently, the constructed input vector for this study comprises a concatenation of 70 dimensions from the grammar features

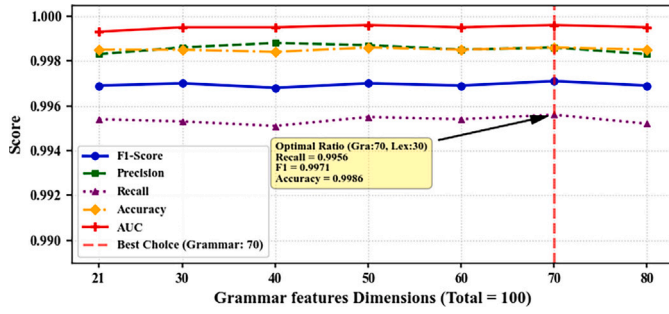


Fig. 10. Conduct feature sensitivity evaluation on Dataset 1.

(derived after dual-channel fusion) and 30 dimensions from the lexical features.

All experiments in this study were conducted on a server equipped with an Intel Core i9-285 K CPU, an NVIDIA GeForce RTX 5090 GPU (32GB), and 128GB of RAM.

We fixed the $Feature_{max}$ of TabDPT at 100. Under this constraint, we conducted a systematic sensitivity analysis to investigate the optimal fusion strategy between grammar features and lexical features, as shown in Fig. 10. Constrained by this fixed total feature dimension, we adjusted the ratio between the two feature types to identify the configuration yielding the optimal balance across performance metrics.

The results indicate highly stable performance across different dimensional allocations, with the F1-score consistently remaining above 0.9960 across the entire tested range. This demonstrates a robust complementarity between deep grammar semantics and explicit lexical features, regardless of minor ratio shifts. While extreme dimensional allocations (e.g., Grammar feature dimension at 21 or 80) show slight fluctuations, the overall metrics plateau at an exceptionally high level.

Crucially, the model reached its optimal performance configuration at a grammar feature dimension of 70 alongside a lexical feature dimension of 30. At this ratio, the model achieves a peak Recall of 0.9956 while maintaining an outstanding F1-score of 0.9971 and an Accuracy of 0.9986. Although the Precision was slightly below the theoretical maximum, it remained highly competitive. To achieve the best comprehensive balance and prioritize the reduction of false negatives, we definitively selected 70:30 as the optimal dimensional ratio for our framework.

5.3. Model results and analysis

As illustrated in Fig. 11 and Table 3, MLF-ICL demonstrates superior performance across all three datasets. Beyond the raw metrics, two critical observations validate the efficacy of our architectural choices:

Robustness on Imbalanced Data (Dataset 1): On the highly imbalanced Dataset 1, MLF-ICL achieves a remarkable F1-score of 99.71%. This significantly outperforms the baseline models. This performance gain is critically attributed to the retrieval-augmented mechanism of TabDPT. Traditional static models tend to bias their decision boundaries toward the majority class (benign URLs) during training. In contrast, our ICL approach constructs a balanced context window by retrieving the nearest malicious neighbors for each query. This process explicitly represents the minority class during inference. This preserves high Recall (99.56%) without sacrificing Precision (99.86%).

Generalization Across Scales (Dataset 2 vs. Dataset 3): The model maintains consistent superiority across both the smaller-scale Dataset 2 and the large-scale Dataset 3. Standard baselines often struggle with overfitting on small data or underfitting on diverse, multi-class data. MLF-ICL's separation of representation learning (via CharBERT) and inference (via TabDPT) allows it to generalize effectively. The CharBERT module provides a stable semantic foundation, while the

TabDPT classifier adapts its reasoning based on the specific distribution of the retrieved context, achieving a specificity near 1.0 (0.9995 and 0.9948, respectively) across both scenarios.

As shown in Table 4, the MLF-ICL framework outperforms state-of-the-art baselines in the majority of metrics across the three datasets. All metric values exceeded 0.98, highlighting the model's exceptional discriminative power and near-perfect AUC scores. On Dataset 1, MLF-ICL achieves consistent gains over close competitors like BiLSTM and Multi-filter CNN, particularly in maintaining a robust balance of precision and recall. This indicates enhanced robustness in detecting positive samples without compromising overall accuracy. Furthermore, the high F1-score underscores the model's significant potential for practical cybersecurity applications where false negatives are costly. These results suggest that the integration of multi-modal features and ICL enables better handling of subtle obfuscations in URL patterns compared to purely CNN models or standard BiLSTM models.

On Datasets 2 and 3, the framework demonstrates outstanding generalization capabilities. It outperforms strong baselines, such as BERT-base + DAPT, kNN models, and ensemble methods like DaE2, across almost all evaluation criteria. While the recall is marginally lower than that of certain highly optimized baselines, this slight trade-off is highly acceptable when viewed alongside the model's exceptional precision and near-perfect AUC scores. In real-world cybersecurity contexts, this equilibrium indicates a strong capability to effectively minimize false positives amid heterogeneous threats without sacrificing overall classification confidence. Across datasets, these findings affirm that MLF-ICL's synergy of deep semantic features, lexical priors, and ICL yields consistent superiority. Moreover, it signals a shift toward more flexible inference paradigms that eliminate the need for retraining in malicious URL detection. This echoes the scaling benefits of real-data pre-training in TabDPT.

This superior performance is attributed to our innovative model architecture, which integrates the Grammar Feature Extraction Module, the Lexical Feature Generation and Selection Module, and the TabDPT Classification Module.

1. **Synergy between Grammar and Lexical Features:** The dominant contribution comes from the Grammar Feature Extraction Module, which provides the semantic and morphological foundation of the framework. Through the CharBERT dual-channel design, HI, and DFF, the model jointly captures word-level intent and character-level obfuscation patterns that are difficult for traditional CNNs or single-channel attention mechanisms to represent. Spatial Dropout further improves robustness, while the pre-trained CharBERT backbone transfers general semantic and morphological knowledge to URL representation learning. On this basis, Lexical Features complement the grammar representation by injecting explicit expert knowledge. Saliency Map-based dynamic weighting highlights high-contribution indicators such as entropy, suspicious symbols, and structural statistics, thereby refining the fused feature space.
2. **Role of the TabDPT Classification Module:** The TabDPT module converts the fused Grammar-Lexical representation into an adaptive inference process rather than a static decision-boundary fitting problem. After the DCG-BERT and M-DWFS modules construct a compact and discriminative tabular vector, TabDPT retrieves relevant historical samples through the FAISS context index and uses them as in-context evidence for classification. This retrieval-augmented mechanism allows the classifier to adjust its reasoning according to the local distribution of each test URL without parameter fine-tuning. Consequently, it mitigates majority-class bias under imbalanced data, improves robustness to concept drift, and explains why the proposed framework maintains strong precision-recall balance across datasets.

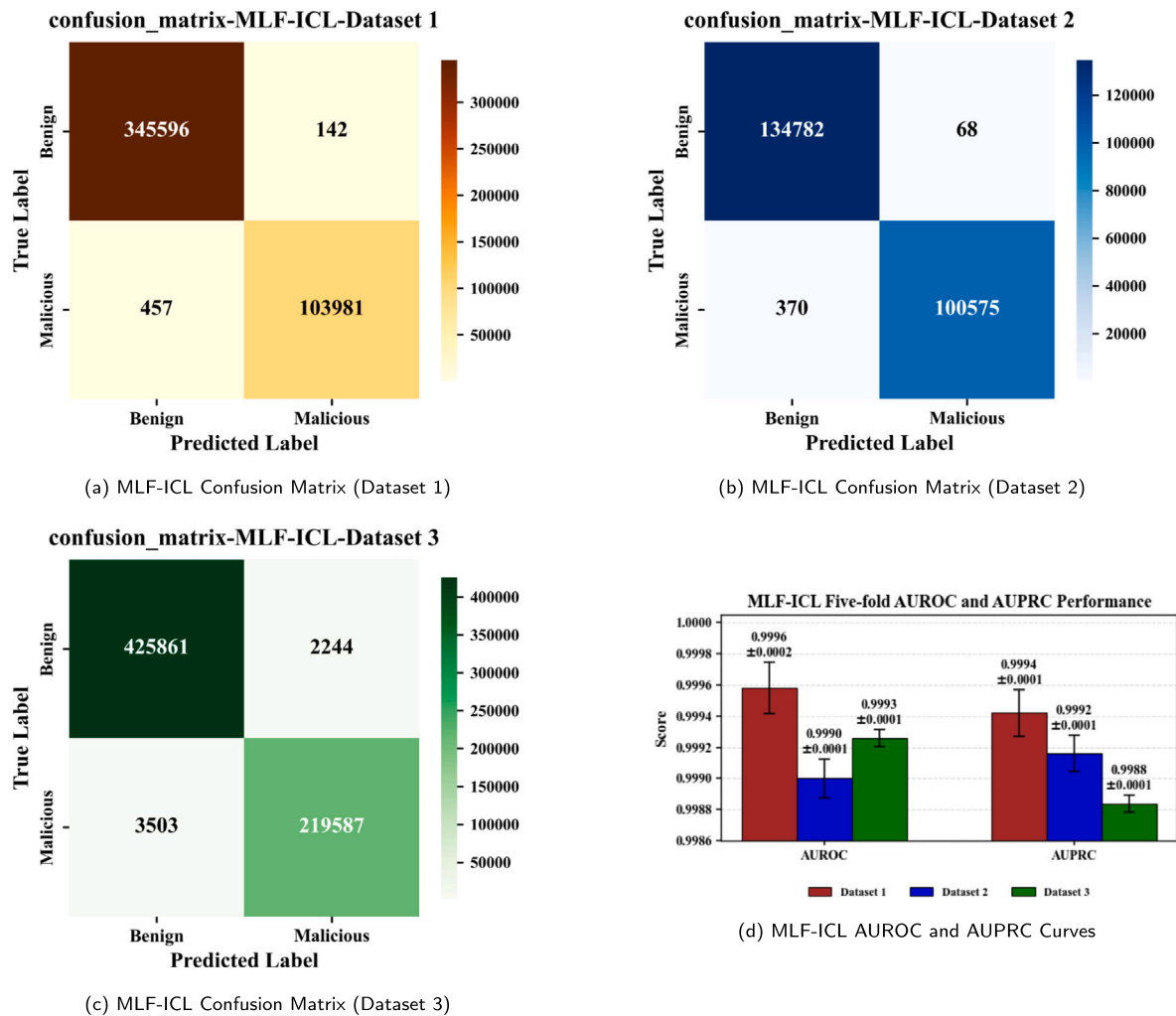


Fig. 11. Visualization of model performance. (a–c) show the confusion matrices from five-fold cross-validation for datasets 1–3, respectively. (d) shows the five-fold cross-validation AUROC and AUPRC.

Table 3

Performance metrics on three datasets.

Dataset	Accuracy	Precision	Recall	F1-score	Specificity
Dataset 1	0.9986 _{±0.0001}	0.9986 _{±0.0003}	0.9956 _{±0.0001}	0.9971 _{±0.0001}	0.9996 _{±0.0001}
	0.9985–0.9987	0.9982–0.9989	0.9954–0.9958	0.9969–0.9973	0.9995–0.9997
Dataset 2	0.9981 _{±0.0002}	0.9994 _{±0.0004}	0.9963 _{±0.0004}	0.9978 _{±0.0002}	0.9995 _{±0.0003}
	0.9979–0.9984	0.9989–0.9999	0.9959–0.9968	0.9975–0.9981	0.9991–0.9998
Dataset 3	0.9912 _{±0.0002}	0.9899 _{±0.0005}	0.9843 _{±0.0004}	0.9871 _{±0.0003}	0.9948 _{±0.0003}
	0.9909–0.9914	0.9892–0.9906	0.9838–0.9848	0.9867–0.9875	0.9944–0.9951

Notes: Subscripts denote SD; second lines are 95% CIs computed over five folds using the t distribution with four degrees of freedom.

5.4. Ablation experiment

To systematically validate the effectiveness of each component and its specific contributions to overall performance, we conducted an ablation study across three independent datasets (Datasets 1, 2, and 3), as illustrated in Figs. 12–14, and Table 5. We evaluated the impact of distinct feature inputs fed into the TabDPT Classifier. As shown in Table 4, the Full Model (integrating both Grammar features after dual-channel Fusion and Lexical Analysis Features) consistently achieved optimal metrics across all test scenarios.

Comparing the configuration using only Lexical Analysis Features against the full model reveals that removing the Grammar features after dual-channel Fusion leads to severe performance degradation, exposing

a critical bottleneck in Recall. For instance, on Dataset 2, the F1-score plummeted from 0.9978 to 0.7899, primarily driven by a drastic drop in Recall to 0.6712. A similarly significant trend is observed on Dataset 3, where the F1-score dropped from 0.9871 to 0.8810. This substantial decline indicates that the Grammar features after dual-channel Fusion serve as the core component. They are responsible for capturing critical discriminative patterns and guaranteeing the model's fundamental classification capability.

Although the configuration utilizing solely Grammar features after dual-channel Fusion established a highly robust baseline, experimental results confirm that Lexical Analysis Features are indispensable for achieving superior performance. Across all three datasets, the

Table 4
Comparison of the proposed MLF-ICL model with existing state-of-the-art methods across three datasets.

Reference	Feature Type	Model	Accuracy	Precision	Recall	Specificity	F1-score	AUROC
<i>Dataset 1</i>								
Afzal et al. [43].(2021)	Wordpiece-level + Lexical Features	LSTM	0.9830	0.9900	0.9770	0.9894	0.9830	0.9935
Roy et al. [44].(2022)	Character-level features	K-Means Clustering	0.9970	0.9700	0.9570	0.9987	0.9680	N/A
Zhang et al. [45].(2024)	Wordpiece-level features	BiLSTM	0.9900	0.9900	0.9900	0.9900	0.9900	N/A
Oznur S et al. [46].(2025)	Wordpiece-level features	Multi-filter CNN	0.9852	0.9218	0.9934	0.9836	0.9563	N/A
Ours	Character + word-based + Lexical features	BiLSTM	0.9932	0.9883	0.9823	0.9965	0.9853	0.9986
		MLF-ICL	0.9986[*]	0.9986[*]	0.9956[*]	0.9996[*]	0.9971[*]	0.9996[*]
		SD	±0.0001	±0.0003	±0.0001	±0.0001	±0.0001	±0.0002
		95% CI	0.9985–0.9987	0.9982–0.9989	0.9954–0.9958	0.9995–0.9997	0.9969–0.9973	0.9994–0.9998
<i>Dataset 2</i>								
Prasad et al. [34].(2024)	Lexical Features, HTML Features	BNB, PA, SGD (80:20)	0.9976	0.9973	0.9986	0.9962	0.9979	N/A
Etem et al. [47].(2024)	Lexical features + t-SNE reduction	kNN	0.9960	0.9960	0.9960	0.9960	0.9950	0.9990
		Decision Tree	0.9970	0.9970	0.9920	0.9989	0.9860	0.9960
Mahdaouy et al. [36].(2025)	Wordpiece-level features	BERT-base + DAPT	0.9980	0.9983	0.9977	0.9983	0.9980	N/A
Ours	Character + word-based + Lexical features	MLF-ICL	0.9981	0.9994[*]	0.9963	0.9995[*]	0.9978	0.9990
		SD	±0.0002	±0.0004	±0.0004	±0.0003	±0.0002	±0.0001
		95% CI	0.9979–0.9984	0.9989–0.9999	0.9959–0.9968	0.9991–0.9998	0.9975–0.9981	0.9989–0.9992
<i>Dataset 3</i>								
Alsaedi et al. [48].(2022)	Lexical + Host-based + Context features	RF + MLP	0.9680	0.9672	0.9688	0.9672	0.9680	N/A
Kibriya et al. [49].(2025)	Character + word-based features	LSTM	0.9808	0.9737	0.9699	0.9864	0.9718	N/A
Li et al. [50].(2025)	Wordpiece-level features	LLM + LSTM	0.9690	0.9690	0.9680	0.9700	0.9680	N/A
Omolaro et al. [51].(2025)	Lexical + Host-based + Context features	DaE2(AdaBoost)	0.9850	0.9777	0.9787	0.9883	0.9800	N/A
		DaE2(bagging)	0.9800	0.9636	0.9564	0.9879	0.9760	N/A
Ours	Character + word-based + Lexical features	MLF-ICL	0.9912[*]	0.9899[*]	0.9843[*]	0.9948[*]	0.9871[*]	0.9993
		SD	±0.0002	±0.0005	±0.0004	±0.0003	±0.0003	±0.0001
		95% CI	0.9909–0.9914	0.9892–0.9906	0.9838–0.9848	0.9944–0.9951	0.9867–0.9875	0.9992–0.9993

Notes: Values for MLF-ICL are five-fold means; SD denotes sample standard deviation; 95% CIs were computed using the t distribution with four degrees of freedom. ^{*} indicates $p < 0.05$ in a one-sided one-sample t -test comparing the five-fold MLF-ICL results with the strongest previously reported point estimate for the same dataset and metric.

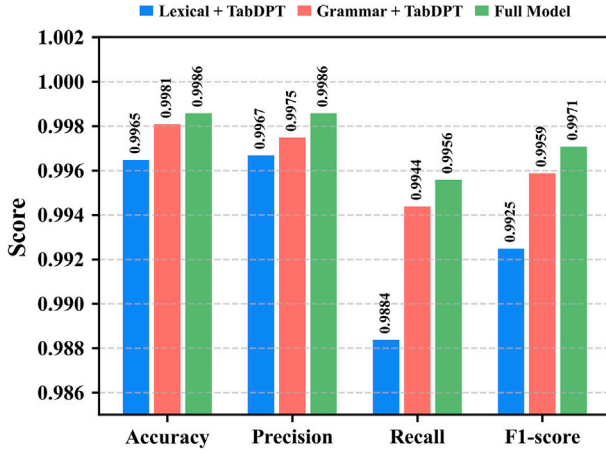


Fig. 12. Comparison of single-modal data feature inputs in Dataset 1.

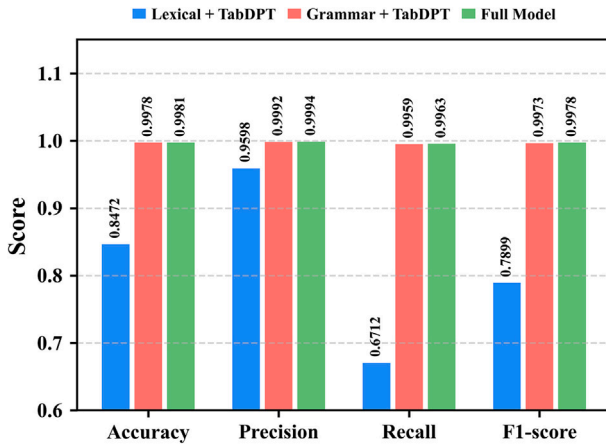


Fig. 13. Comparison of single-modal data feature inputs in Dataset 2.

integration of Lexical Analysis Features led to further convergence and improvement in Accuracy and F1 scores. Specifically, on Dataset 1, the F1-score increased from 0.9959 to 0.9971, and on Dataset 2, it increased from 0.9973 to 0.9978, with consistent positive gains across all metrics.

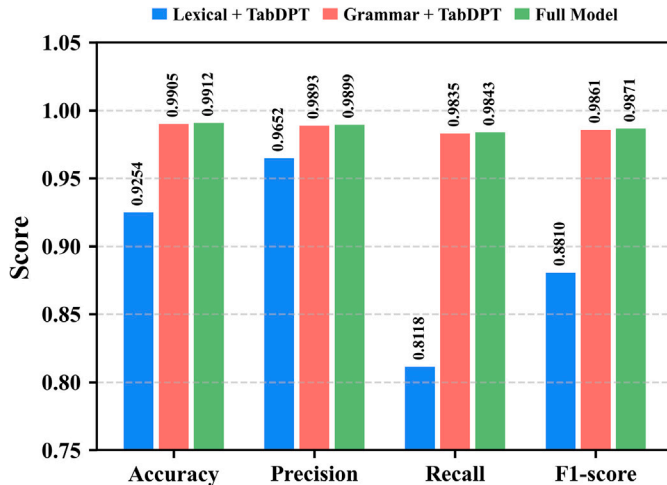


Fig. 14. Comparison of single-modal data feature inputs in Dataset 3.

This demonstrates that Lexical Analysis Features effectively enhance the model's ability to handle boundary or complex samples. Consequently, they serve as a critical element for elevating the model's generalization limit.

In conclusion, the Grammar features after dual-channel Fusion guarantee the performance baseline (lower bound), while the Lexical Analysis Features elevate the performance ceiling (upper bound). Their synergistic interaction within the TabDPT classifier enables the framework to achieve optimal performance across diverse data distributions.

5.5. Cross-dataset evaluation on recent and evolving threats

To rigorously address the rapid evolution of cyberattacks and demonstrate model adaptability to contemporary and novel threats, our experimental design is structured into two main phases. Following the baseline evaluations, we leverage Dataset 1 as the foundational knowledge base for a strict evaluation across external datasets. We selected this dataset because it demonstrated the best local performance. Specifically, we freeze the DCG-BERT backbone previously trained on Dataset 1 and construct the TabDPT context index exclusively using the training set of D1. We then directly evaluate this locked framework on two strictly unseen external targets. First, we test on Dataset 2, selected for its highly recent phishing domains, to verify effectiveness against contemporary attacks. Second, we utilize Dataset 4, which comprises diverse attack vectors such as DNS Tunneling and DGA. This explicitly demonstrates the model's ability to identify novel attacks using an index built from historical data, as shown in Table 6.

Generalization to Newer Phishing Threats (Test on Dataset 2):

On Dataset 2, our model achieves competitive results, with key metrics exceeding 0.96. In contrast, recent evaluations of baseline models in similar cross-dataset scenarios, such as XGBoost and BERT, yielded F1-scores of approximately 0.69 and 0.83, respectively. This suggests that, despite being trained on earlier datasets, MLF-ICL's multi-modal feature fusion effectively captures shared malicious patterns in newer phishing URLs.

Robustness Against Novel Threat Structures (Test on Dataset 4):

We further evaluated the model's boundary performance on Dataset 4, which includes novel threats such as DNS tunneling and Domain Generation Algorithms (DGA). The structural patterns of these URLs differ significantly from the semantics-focused phishing patterns in our training data. Specifically, DGAs exhibit high-entropy randomness, and DNS tunnels feature long, encoded strings. Consequently, the model's precision faced anticipated challenges, resulting in a lower F1-score of 0.6617.

However, despite this structural domain shift, the model maintained a high recall of 0.9879. In practical cybersecurity operations, minimizing false negatives is often prioritized. The model faces challenges in precisely delineating classification boundaries for entirely unfamiliar, machine-generated syntax. Nevertheless, this sustained high recall indicates that its feature extraction mechanism remains highly sensitive to underlying structural anomalies. This demonstrates that MLF-ICL retains a reliable threat-awareness capability even when confronted with non-semantic, novel attacks.

5.6. Computational cost

Table 7 summarizes the parameter counts and memory footprint across different datasets. Regardless of dataset variations, the structural parameters of the model remain constant. The primary variable overhead originates from the memory required for the FAISS index. This memory requirement increases alongside the size of the context pool.

Regarding computational complexity, the architecture involves multiple modules. This inevitably introduces a higher initial overhead. To address this issue in practical deployments, a strategy of decoupling online and offline operations can be adopted. Specifically, the computationally demanding DCG-BERT feature extraction module can be

Table 5Ablation study of different feature module combinations on three datasets (mean $\pm$ SD; 95% CI for the full model).

Dataset	Model	Accuracy	Precision	Recall	Specificity	F1-score
Dataset 1	Lexical Analysis Features + TabDPT	0.9965	0.9967	0.9884	0.9990	0.9925
	SD	± 0.0002	± 0.0005	± 0.0005	± 0.0001	± 0.0005
	Grammar Features + TabDPT	0.9981	0.9975	0.9944	0.9992	0.9959
	SD	± 0.0002	± 0.0005	± 0.0005	± 0.0002	± 0.0004
	Full Model (Hybrid)	0.9986*	0.9986*	0.9956*	0.9996*	0.9971*
	SD	± 0.0001	± 0.0003	± 0.0001	± 0.0001	± 0.0001
95% CI		0.9985–0.9987	0.9982–0.9989	0.9954–0.9958	0.9995–0.9997	0.9969–0.9973
Dataset 2	Lexical Analysis Features + TabDPT	0.8472	0.9598	0.6712	0.9790	0.7899
	SD	± 0.0010	± 0.0012	± 0.0020	± 0.0007	± 0.0017
	Grammar Features + TabDPT	0.9978	0.9992	0.9959	0.9993	0.9973
	SD	± 0.0002	± 0.0003	± 0.0004	± 0.0002	± 0.0003
	Full Model (Hybrid)	0.9981*	0.9994	0.9963	0.9995	0.9978*
	SD	± 0.0002	± 0.0004	± 0.0004	± 0.0003	± 0.0002
95% CI		0.9979–0.9984	0.9989–0.9999	0.9959–0.9968	0.9991–0.9998	0.9975–0.9981
Dataset 3	Lexical Analysis Features + TabDPT	0.9254	0.9652	0.8118	0.9847	0.8810
	SD	± 0.0008	± 0.0004	± 0.0024	± 0.0004	± 0.0014
	Grammar Features + TabDPT	0.9905	0.9893	0.9835	0.9943	0.9861
	SD	± 0.0003	± 0.0004	± 0.0006	± 0.0004	± 0.0005
	Full Model (Hybrid)	0.9912*	0.9899*	0.9843*	0.9948*	0.9871*
	SD	± 0.0002	± 0.0005	± 0.0004	± 0.0003	± 0.0003
95% CI		0.9909–0.9914	0.9892–0.9906	0.9838–0.9848	0.9944–0.9951	0.9867–0.9875

Notes: SD denotes standard deviation. The 95% confidence intervals were computed using the t distribution with four degrees of freedom. * indicates $p < 0.05$ in a one-sided Welch's t -test comparing the full model with the strongest non-full ablation variant for the same dataset and metric.

Table 6

Cross-dataset performance comparison of baseline models and our proposed method.

Reference	Model	Accuracy	Precision	Recall	Specificity	F1-score	ROC-AUC
Misiek et al. [52].(2026)	XGBoost (Test on Dataset 2)	0.7216	0.6548	0.7376	0.7097	0.6937	N/A
Alorvor et al. [53].(2025)	BiLSTM (Test on Dataset 2)	0.7600	0.8300	0.7600	0.7600	0.7400	0.9600
	TCN (Test on Dataset 2)	0.8100	0.8600	0.8100	0.8100	0.8000	0.9700
	BERT (Test on Dataset 2)	0.8400	0.8800	0.8400	0.8400	0.8300	0.9400
Ours	MLF-ICL (Test on Dataset 2)	0.9843* ± 0.0006 [0.9834,0.9853]	0.9955* ± 0.0041 [0.9890,1.0000]	0.9679* ± 0.0053 [0.9594,0.9764]	0.9967* ± 0.0030 [0.9918,1.0000]	0.9815* ± 0.0008 [0.9802,0.9827]	0.9927* ± 0.0017 [0.9900,0.9954]
	MLF-ICL (Test on Dataset 4)	0.4949 ± 0.0059 [0.4855,0.5043]	0.4974 ± 0.0030 [0.4926,0.5022]	0.9879 ± 0.0143 [0.9652,1.0000]	0.0019 ± 0.0038 [0.0000,0.0080]	0.6617 ± 0.0058 [0.6524,0.6710]	0.3566 ± 0.0986 [0.1997,0.5135]

Proposed rows report mean $\pm$ SD with 95% confidence intervals in brackets. * $p < 0.05$ in a one-sided one-sample t -test comparing Proposed (Test on Dataset 2) with the strongest reported literature baseline for the same metric.

Table 7

Performance and overhead metrics across different datasets.

Dataset	Preprocessing Params (M)	TabDPT Params (M)	Preprocessing Time (ms)	Retrieval Memory (MB)	Build Time (s)	Inference Speed (samples/s)	End-to-End Time (ms)
Dataset 1	116.27	76.87	8.07	274.77	0.89	5.30	196.75
Dataset 2	116.27	76.87	7.86	143.92	0.40	5.01	207.46
Dataset 3	116.27	76.87	8.13	397.45	1.12	6.67	158.06
Dataset 4	116.27	76.87	8.15	274.77	0.88	6.76	156.08

deployed on backend or cloud servers. Meanwhile, the TabDPT engine handles classification tasks online. Experiments indicate that feature extraction takes approximately 8.00 milliseconds per URL, and inferring and classifying each URL takes about 180 milliseconds. Conversely, updating the threat knowledge base by reconstructing the FAISS index requires approximately 0.8 s. Compared to the lengthy retraining time demanded by traditional models, this rapid adaptation capability effectively mitigates the issue of detection lag. The framework demonstrates significant advantages in detection accuracy and handling complex obfuscations. Therefore, the FAISS memory overhead serves as a reasonable compromise in critical domains such as finance and cybersecurity.

5.7. Limitations and future work

Although the proposed MLF-ICL framework achieves excellent classification performance on the benchmark datasets, certain limitations remain. In this subsection, we briefly discuss these constraints:

- While the framework exhibits robust detection capabilities against novel attacks via ICL, maintaining this capability requires continuously adding new attack samples to the external context pool. The rigorous data preprocessing and periodic representation updates required to curate these new threat features introduce ongoing maintenance overhead.
- The architecture utilizing retrieval relies on a continuously growing FAISS index, which introduces notable memory consumption. This storage footprint poses significant constraints when attempting to deploy the model on edge devices with limited resources.
- The framework remains less effective for structurally heterogeneous and weakly semantic attack families, such as DGA-based domains and DNS tunneling. These samples are driven more by algorithmic randomness, encoded payloads, and protocol-specific structures than by phishing-oriented lexical or brand semantics.

To address these limitations, our future work will focus on:

- Developing automated and lightweight data curation pipelines to efficiently update the external knowledge index, ensuring continuous adaptation to new threats with minimal preprocessing overhead.
- Exploring index compression strategies and retrieval techniques optimized for memory to minimize the storage footprint, thereby enabling feasible and robust deployment on edge devices, where adaptive IoT anomaly detection has also shown the value of lightweight feature-enriched models [54].
- Enhancing cross-family adaptability to DGA-based domains and DNS tunneling. Specifically, we will incorporate representative samples into the external context pool, design entropy-, n-gram-, and protocol-aware lexical features, and explore domain-adaptive retrieval and calibration strategies for severe out-of-distribution shifts.

6. Conclusion

In this study, we addressed a critical limitation of traditional deep learning models in malicious URL detection. Specifically, we tackled the rigidity of fixed decision boundaries in the face of rapid concept drift. We proposed **MLF-ICL**, a novel framework that shifts the paradigm from “static classification” to “adaptive inference” by synergizing multi-modal feature fusion with Tabular Foundation Models. The core findings and contributions are summarized as follows:

1. **Deep Semantic Decoding:** We demonstrated that our enhanced DCG-BERT architecture effectively bridges the semantic gap between natural language and agrammatical URL strings. By integrating Multi-order Dynamic Feature Fusion (DFF) and Context Gating, the model precisely captures morphological obfuscations (e.g., homograph attacks) that typically evade standard BERT models.
2. **Synergistic Feature Complementarity:** We validated the necessity of fusing implicit deep representations with explicit expert knowledge. The Saliency Map-based selection mechanism ensures that high-value lexical indicators (such as entropy anomalies in DGA domains) are injected into the feature space. This compensates for the black-box limitations of pure deep learning approaches.
3. **Superiority of In-Context Learning:** We pioneered the application of TabDPT as a retrieval-augmented classifier. Experimental results confirm that this ICL paradigm significantly outperforms static baselines. It achieves F1-scores of 99.71%, 99.78%, and 98.71% across imbalanced and multi-class datasets. Crucially, the retrieval mechanism effectively mitigates class imbalance bias by dynamically constructing balanced context windows.

This study offers three critical implications for the cybersecurity community. First, regarding feature extraction, our methodology achieves a highly comprehensive and discriminative representation of malicious URLs by dynamically fusing implicit deep semantics with explicit expert rules. Second, traditional machine learning models are constrained by static parameters. In contrast, our TabDPT framework requires no training and eliminates detection lag. This allows systems to instantly adapt to novel threats via simple index updates rather than costly retraining. Finally, this paradigm also exhibits relatively good performance when facing cross-dataset scenarios and other forms of URL threats.

CRedit authorship contribution statement

Lan Liu: Writing – review & editing, Writing – original draft, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Fengwei Guo:** Writing – review & editing, Writing

– original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Weijie Liang:** Writing – original draft, Visualization, Validation, Software, Methodology, Investigation. **Kundi Yao:** Writing – review & editing, Supervision, Project administration.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Lan Liu reports that financial support was provided by Guangdong Province Department of Education. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the Scientific Research Projects of Guangdong Province Department of Education, Specialized in Key Areas (2022ZDZX1015).

Data availability

Data will be made available on request.

References

- [1] D. Jampen, B. Janet, R.J.A. Kumar, Malicious URL detection: a comparative study, in: 2021 International Conference on Artificial Intelligence and Smart Systems (ICAIS), IEEE, 2021, pp. 1147–1151.
- [2] M. Alawida, A.E. Omolara, O.I. Abiodun, M. Al-Rajab, A deeper look into cybersecurity issues in the wake of COVID-19: a survey, J. King Saud Univ. Comput. Inf. Sci. 34 (10) (2022) 8176–8206.
- [3] Anti-Phishing Working Group (APWG), Phishing Activity Trends Report, 1st Quarter 2025. Technical report, APWG, 2025.
- [4] Federal Bureau of Investigation (FBI), Internet Crime Report 2024. Technical report, Internet Crime Complaint Center (IC3), 2025. Cited in APWG Phishing Activity Trends Report Q3 2025.
- [5] A. Basit, M. Zafar, X. Liu, A.R. Javed, Z. Jalil, K. Kifayat, A comprehensive survey of AI-enabled phishing attacks detection techniques, Telecommun. Syst. 76 (2021) 139–154.
- [6] M.S.I. Mamun, M.A. Rathore, A.H. Lashkari, N. Stakhanova, A.A. Ghorbani, Detecting malicious URLs using lexical analysis, in: International Conference on Network and System Security, Springer, Cham, 2016, pp. 467–482.
- [7] H. Le, Q. Pham, D. Sahoo, S.C. Hoi, URLNet: learning a URL representation with deep learning for malicious URL detection, arXiv preprint arXiv:1802.03162, 2018.
- [8] A.S. Bozkir, F.C. Dalgic, M. Aydos, GramBeddings: a new neural network for URL based identification of phishing web pages through n-gram embeddings, Comput. Secur. 124 (2023) 102964.
- [9] M. Naseer, F. Ullah, S. Saeed, et al., Explainable TabNet ensemble model for identification of obfuscated URLs with features selection to ensure secure web browsing, Sci. Rep. 15 (1) (2025) 9496.
- [10] Z. Li, C. Huang, X. Jia, MFFURL: multi-modal feature fusion-based approach for malicious URL detection, Comput. Netw. (2025) 111898.
- [11] R. Liu, Y. Wang, Z. Guo, et al., TransURL: improving malicious URL detection with multi-layer transformer encoding and multi-scale pyramid features, Comput. Netw. 253 (2024) 110707.
- [12] R. Liu, Y. Wang, H. Xu, et al., PMANet: malicious URL detection via post-trained language model guided multi-level feature attention network, Inf. Fusion 113 (2025) 102638.
- [13] W. Ma, Y. Cui, C. Si, et al., CharBERT: character-aware pre-trained language model, arXiv preprint arXiv:2011.01513, 2020.
- [14] N. Hollmann, S. Müller, L. Purucker, et al., Accurate predictions on small data with a tabular foundation model, Nature 637 (8045) (2025) 319–326.
- [15] V. Thomas, J. Ma, R. Hosseinzadeh, et al., Retrieval & fine-tuning for in-context tabular models, in: Advances in Neural Information Processing Systems (NeurIPS), vol. 37, 2024, pp. 108439–108467.
- [16] J. Ma, V. Thomas, R. Hosseinzadeh, et al., TabDPT: scaling tabular foundation models, arXiv preprint arXiv:2410.18164, 2024.
- [17] A. Ozcan, C. Catal, E. Donmez, et al., A hybrid DNN-LSTM model for detecting phishing URLs, Neural Comput. Appl. 35 (7) (2023) 4957–4973.
- [18] D.K. McGrath, M. Gupta, Behind phishing: an examination of phisher modus operandi, in: Proceedings of the 1st USENIX Workshop on Large-Scale Exploits and Emergent Threats (LEET), USENIX Association, 2008, pp. 4.
- [19] J. Ma, et al., Identifying suspicious URLs: an application of large-scale online learning, in: Proceedings of the 26th Annual International Conference on Machine Learning (ICML), ACM, 2009.
- [20] J. Ma, et al., Beyond blacklists: learning to detect malicious web sites from suspicious URLs, in: Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, ACM, 2009.

- [21] R.S. Rao, T. Vaishnavi, A.R. Pais, CatchPhish: detection of phishing websites by inspecting URLs, *J. Ambient Intell. Humaniz. Comput.* 11 (2020) 813–825.
- [22] O.K. Sahingoz, E. Buber, O. Demir, B. Diri, Machine learning based phishing detection from URLs, *Expert Syst. Appl.* 117 (2019) 345–357.
- [23] M. Cherradi, H. El Mahajer, Malicious URL detection using machine learning techniques, *International Journal of Data Informatics and Intelligent Computing* 4 (2) (2025) 41–52.
- [24] Y. Huang, J. Qin, W. Wen, Phishing URL detection via capsule-based neural network, in: 2019 IEEE 13th International Conference on Anti-Counterfeiting, Security, and Identification (ASID), IEEE, 2019, pp. 22–26.
- [25] H.-H. Wang, L. Yu, S.-W. Tian, Y.-F. Peng, X.-J. Pei, Bidirectional LSTM malicious webpages detection algorithm based on convolutional neural network and independent recurrent neural network, *Appl. Intell.* 49 (2019) 3016–3026.
- [26] W. Chang, F. Du, Y. Wang, Research on malicious URL detection technology based on BERT model, in: 2021 IEEE 9th International Conference on Information, Communication and Networks (ICIN), IEEE, 2021, pp. 340–345.
- [27] P. Maneriker, J.W. Stokes, E.G. Lazo, D. Carutasu, F. Tajaddodianfar, A. Gururajan, URLTran: improving phishing URL detection using transformers, in: MILCOM 2021 - 2021 IEEE Military Communications Conference (MILCOM), IEEE, 2021, pp. 197–204.
- [28] Y. Wang, W. Ma, H. Xu, Y. Liu, P. Yin, A lightweight multi-view learning approach for phishing attack detection using transformer with mixture of experts, *Appl. Sci.* 13 (13) (2023) 7429.
- [29] P. Xu, A transformer-based model to detect phishing URLs, *arXiv preprint arXiv:2109.02138*, 2021.
- [30] A. Tanna, P. Seth, M. Bouadi, et al., TabTune: a unified library for inference and fine-tuning tabular foundation models, *arXiv preprint arXiv:2511.02802*, 2025.
- [31] G. Chen, Z. Yang, Risk prediction for gastrointestinal bleeding in pediatric Henoch-Schönlein purpura using an interpretable transformer model, *Front. Physiol.* 16 (2025) 1630807.
- [32] S. Ruiz-Villafranca, J. Roldán-Gómez, J. Carrillo-Mondejar, et al., WFE-Tab: overcoming limitations of TabPFN in IIoT-MEC environments with a weighted fusion ensemble-TabPFN model for improved IDS performance, *Future Gener. Comput. Syst.* 166 (2025) 107707.
- [33] J. Ma, V. Thomas, R. Hosseinzadeh, H. Kamkari, A. Labach, J.C. Cresswell, K. Golestan, G. Yu, M. Volkovs, A.L. Caterini, TabDPT: scaling tabular foundation models, *arXiv preprint arXiv:2410.18164*, 2024.
- [34] A. Prasad, S. Chandra, PhiUSIIL: a diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning, *Comput. Secur.* 136 (2024) 103545.
- [35] Y. Bubnov, DNS tunneling queries for binary classification, Mendeley Data, Version 1, 2019, <https://doi.org/10.17632/mzn9hvdxcg.1>.
- [36] M. Zago, M. Gil Pérez, G. Martínez Pérez, UMUDGA: a dataset for profiling algorithmically generated domain names in botnet detection, *Data Br.* 30 (2020) 105400.
- [37] T.A. Tuan, N.V. Anh, T.T. Luong, H.V. Long, Utl_dga22-a dataset for DGA botnet detection and classification, *Comput. Netw.* 221 (2023) 109508.
- [38] A. El Mahdaoui, S. Lamsiyah, M. Janati Idrissi, et al., DomURLs BERT: pre-trained BERT-based model for malicious domains and URLs detection and classification, *J. Netw. Syst. Manag.* 34 (2) (2026) 36.
- [39] G. Jawahar, B. Sagot, D. Seddah, What does BERT learn about the structure of language? in: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL), Association for Computational Linguistics, 2019.
- [40] X. Ouyang, K. Gu, P. Zhou, Spatial pyramid pooling mechanism in 3D convolutional network for sentence-level classification, *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 26 (11) (2018) 2167–2179.
- [41] M. Hussain, C. Cheng, R. Xu, et al., CNN-fusion: an effective and lightweight phishing detection method based on multi-variant ConvNet, *Inf. Sci.* 631 (2023) 328–345.
- [42] X. Sun, Y. Liu, M. Xu, et al., Feature selection using dynamic weights for classification, *Knowl.-Based Syst.* 37 (2013) 541–549.
- [43] S. Afzal, M. Asim, A.R. Javed, et al., URLDeepDetect: a deep learning approach for detecting malicious URLs using semantic vector models, *J. Netw. Syst. Manag.* 29 (3) (2021) 21.
- [44] S.S. Roy, A.I. Awad, L.A. Amare, et al., Multimodal phishing URL detection using LSTM, bidirectional LSTM, and GRU models, *Future Internet* 14 (11) (2022) 340.
- [45] L. Zhang, Q. Yan, Detect malicious websites by building a neural network to capture global and local features of websites, *Comput. Secur.* 137 (2024) 103641.
- [46] Ö Akçam, A. Tekerek, M. Tekerek, Development of BiLSTM deep learning model to detect URL-based phishing attacks, *Comput. Electr. Eng.* 123 (2025) 110212.
- [47] T. Etem, M. Teke, Advanced phishing detection: leveraging t-SNE feature extraction and machine learning on a comprehensive URL dataset, *Acta Infologica* 8 (2) (2024) 213–221.
- [48] M. Alsaedi, F.A. Ghaleb, F. Saeed, et al., Cyber threat intelligence-based malicious URL detection model using ensemble learning, *Sensors* 22 (9) (2022) 3373.
- [49] H. Kibriya, R. Amin, S.S. Alshamrani, et al., Lightweight malicious URL detection using deep learning and large language models, *Sci. Rep.* 15 (2025) 43044.
- [50] Z. Li, S. Zhuo, J. He, et al., Behavior-enhanced representation learning for user behavior analysis, *IEEE Trans. Inf. Forensics Secur.* 20 (2025) 9275–9288.
- [51] A.E. Omolara, M. Alawida, DaE2: unmasking malicious URLs by leveraging diverse and efficient ensemble machine learning for online security, *Comput. Secur.* 148 (2025) 104170.
- [52] M. Misiek, T. Hyla, XGBoost-based URL phishing detection method with cross-dataset validation, *IEEE Access* 14 (2026) 43200–43212.
- [53] M.L.E. Alorvor, S. Dadkhah, Real-time phishing detection for brand protection using temporal convolutional network-driven URL sequence modeling, *Electronics* 14 (18) (2025) 3746.
- [54] P. Gujjar, R.M. Devadas, K. Nikitha, Wavelet-based intelligent framework for network traffic anomaly detection in IoT embedded systems, *International Journal of Data Informatics and Intelligent Computing* 5 (1) (2026) 1–10.

Author biography



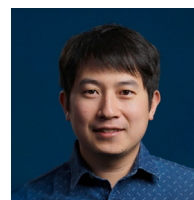
Lan Liu was born in YiYang, China, in 1977. She received the B.S., M.S., and Ph.D. degrees in computer architecture from Huazhong University of Science and Technology, Wuhan, Hubei, China, in 1999, 2002, and 2007, respectively. She joined Guangdong Polytechnic Normal University as an Assistant in 2003. She became a Professor and a Master Tutor of the Department of Electronic Information in 2022 and 2015, respectively. She was a Visiting Professor conducting research on network security in the Computer Science Laboratory, University of Waikato, Hamilton, New Zealand, in 2016. She is the author of more than 30 articles published in IEEE conferences and journals (EI- or SCI-indexed). Her research interests include network security, deep learning, and software-defined networking.



Fengwei Guo was born in Maoming, China. He received the B.S. degree in Network Engineering from Guangdong University of Science and Technology in 2024. He is currently pursuing the M.S. degree in Network and Information Security at Guangdong Polytechnic Normal University. His main research interests include anomaly intrusion detection systems and deep learning.



Weijie Liang was born in Guangzhou, China. He received the B.S. degree in Computer Science and Technology from Guangzhou Xinhua University in 2024. He is currently pursuing the M.S. degree in Network and Information Security at Guangdong Polytechnic Normal University. His main research interests include deep learning and anomaly detection systems.



Kundi Yao is an Assistant Professor at Ontario Tech University. He received his Ph.D. in Software Engineering from Queen's University and his Master's degree from Concordia University. His research interests include AIOps, log analytics, software performance, and big data-driven software engineering, with the goal of improving the quality, observability, and maintainability of large-scale software systems. His research has been adopted in industry through collaboration with partners such as BlackBerry and ERA Environmental. Contact him at <https://kdyao.github.io/>.